CHAPTER 1

INTRODUCTION

1.1 Background

The transition from academic education to professional employment has become increasingly complex in the modern digital economy. Students graduating from engineering and technical institutions are expected not only to possess theoretical understanding of their subjects but also to demonstrate communication ability, interview readiness, aptitude, problem-solving capability, and awareness of industry expectations. In many cases, students are technically capable but remain underprepared for actual placement processes because the available preparation resources are fragmented, inconsistent, or difficult to follow in a structured way.

Traditional career preparation methods generally depend on classroom guidance, handwritten notes, scattered online videos, informal peer groups, and generic interview question lists. While these approaches may offer partial support, they often fail to provide a continuous and personalized preparation journey. Students may know that they need to improve, but they frequently do not know where to begin, what to study first, how to track progress, or how to connect academic learning with real-world recruitment expectations. This creates confusion, reduces confidence, and often results in inefficient preparation during crucial placement periods.

At the same time, digital platforms have transformed how users consume knowledge, develop skills, and prepare for professional opportunities. Web applications now support adaptive learning, structured user journeys, personalized recommendations, and real-time engagement in ways that traditional preparation methods cannot easily match. With the growing adoption of artificial intelligence in education and career support, it has become possible to build platforms that not only present information but also guide users through decision-making, resource discovery, and preparation workflows in a more intelligent and user-centric way.

The present project, titled **AI-Powered Career Guidance and Interview Preparation Platform**, is conceived in response to this need. It aims to create a web-based system that combines structured career guidance, interview preparation resources, onboarding, authentication, and a clear user experience into a single coherent platform. The project is designed primarily for engineering students and early-career learners who require a focused, accessible, and scalable digital environment to prepare for interviews and career growth.

The background of this work is therefore rooted in two converging realities. First, students increasingly need guided and personalized preparation support to bridge the gap between

academic study and professional expectations. Second, modern web technologies such as Next.js, React, authentication frameworks, and modular UI systems provide a practical foundation for building such a platform in a scalable and maintainable way. This project brings these two realities together by designing a structured platform intended to simplify the early stages of career readiness and improve the quality of interview preparation.

1.2 Problem Context and Motivation

One of the most common challenges faced by students during placement preparation is the lack of a unified environment for career development. Academic institutions generally focus on conceptual learning, examinations, and project work, while recruitment processes assess communication, confidence, practical problem-solving, and applied understanding. As a result, many students experience a disconnect between what they study in classrooms and what companies expect in actual interviews. This disconnect becomes even more visible for learners who do not have access to mentors, career counselors, or structured preparation communities.

The current ecosystem of career preparation resources is highly fragmented. A learner may use one platform for aptitude questions, another for coding problems, a third for interview experiences, a fourth for resume templates, and social media channels for informal advice. Although each resource may be useful in isolation, the lack of integration creates friction. Students must manually assemble their own preparation workflow, decide priorities independently, and evaluate resource quality on their own. This fragmentation often leads to wasted time, duplicated effort, inconsistent preparation, and anxiety during placement seasons.

Another important issue is the absence of guided onboarding and contextual navigation in many existing systems. Most generic learning or interview websites assume that users already know what they need. In reality, many students begin their preparation with uncertainty. They may not know whether to start with aptitude, technical interview questions, communication improvement, or industry-specific preparation. A well-designed platform should not simply provide content; it should guide the user from awareness to action. This need for guided progression strongly motivates the design approach of the current project, where the homepage itself acts as a structured narrative leading the user through platform value, steps, trust elements, and eventual action.

The motivation behind this project is therefore both practical and academic. Practically, the system seeks to reduce confusion for students by offering a centralized and user-friendly preparation platform. Academically, it demonstrates how software engineering principles can be applied to a meaningful real-world problem. The project incorporates requirements analysis, system design, database modeling, authentication flow, UI structuring, and testing methodology to develop a complete academic software project with real-world relevance.

There is also a strong motivational link to the growing importance of AI-supported educational systems. As artificial intelligence becomes more integrated into learning environments, there is increasing value in building platforms that can eventually support intelligent recommendations,

personalized question generation, adaptive feedback, and data-driven career guidance. Although the current version focuses on core platform structure rather than advanced AI implementation, it is intentionally designed to remain extensible so that richer AI-powered features can be integrated in later versions without disrupting the basic architecture.

Thus, the motivation for this project arises from a combination of educational need, placement pressure, fragmented existing resources, and the opportunity created by modern web technologies. The project aims to serve as a bridge between students' aspirations and the digital systems capable of supporting those aspirations in a practical, scalable, and structured manner.

1.3 Problem Statement

Students and early-career professionals often lack access to a unified, intuitive, and interactive platform that combines career guidance, interview preparation, structured navigation, and personalized progression support within a single digital environment. Most existing resources are either too generic, too fragmented, or too narrowly focused on one specific aspect of preparation, such as coding questions, mock tests, or course delivery. As a result, users frequently struggle to determine how to begin their preparation, what sequence to follow, which content is relevant to their goals, and how to sustain consistency throughout the process.

The absence of such a platform creates several practical difficulties. Students may spend significant time moving between multiple websites, comparing content, repeating topics unnecessarily, or overlooking important preparation areas such as communication, onboarding readiness, and role-specific interview expectations. In addition, the lack of centralized organization can reduce engagement and weaken user confidence, especially when preparation is carried out without mentorship or institutional support.

The problem addressed by this project can therefore be stated as follows: **How can a web-based platform be designed and implemented to guide users through clearly defined career preparation steps while providing structured interview resources, AI-oriented extensibility, secure access control, and a user-friendly interface within one integrated system?**

This problem statement defines the central focus of the project. It does not merely ask how to build a website containing interview questions; rather, it asks how to design an integrated user experience that supports learning, preparation, and navigation in a more intelligent and coherent way. The emphasis is on creating a platform that combines usability, modular design, and scalability while addressing a real need among engineering students preparing for placements and early professional opportunities.

1.4 Objectives

The primary objective of this project is to design and develop a responsive, modern, and structured web platform that supports career guidance and interview preparation for students and early-career professionals. The platform is intended to provide clarity, accessibility, and guided interaction so

that users can move from initial curiosity to meaningful preparation through a coherent digital workflow.

The specific objectives of the project are as follows:

- To design and implement a user-friendly web platform dedicated to career guidance and interview preparation.
- To provide structured platform sections such as features, preparation steps, FAQs, and testimonials that clearly communicate value to the user.
- To integrate secure authentication and onboarding so that the platform can personalize user flow based on sign-in and profile completion status.
- To create intuitive navigation and call-to-action mechanisms that direct users from the landing page to onboarding and dashboard access.
- To follow formal software engineering practices including requirements analysis, UML-based thinking, database design, and systematic testing.
- To ensure that the platform architecture remains modular and extensible for future integration of additional AI features such as resume analysis, question recommendation, and adaptive feedback.

Beyond these direct implementation goals, the project also aims to demonstrate sound academic and engineering discipline. It seeks to show that modern web technologies can be used not only for interface development but also for solving meaningful educational and career-related problems. In that sense, the objectives of the project are both product-oriented and learning-oriented: the final system serves users, while the development process reflects software design principles and documentation rigor.

1.5 Scope of the Project

The scope of the project is focused on the design and implementation of the client-facing web application and its core support logic for guided career preparation. The platform includes the major user interface sections identified in the source report, namely the hero section, features section, statistics section, “How It Works” section, testimonials section, FAQ section, authentication flow, onboarding path, and dashboard routing logic. These components together create a complete foundational experience that demonstrates the intended user journey from entry to engagement.

Within this scope, the project addresses usability, structured navigation, secure sign-in flow, and modular interface development. It also includes underlying academic deliverables such as requirements analysis, software architecture description, database design, testing strategy, and role allocation. This means the project is not limited to interface appearance alone; it also includes the planning and documentation required for a full academic software engineering report.

However, the current scope is intentionally bounded. The project does not attempt to build a complete enterprise recruitment platform, a full learning management system, or a production-scale AI recommendation engine. Advanced functionalities such as payment integration, large-scale analytics, fully automated resume parsing, content administration dashboards, or highly personalized live AI interview coaching remain outside the current implementation boundary. These are treated as potential future enhancements rather than core deliverables of the present version.

This carefully defined scope is important because it ensures the project remains realistic, focused, and academically manageable. By limiting the scope to a strong foundational system, the project is able to deliver a coherent and polished implementation while still leaving room for future expansion. In effect, the current system can be understood as a scalable starting framework for a more advanced career guidance ecosystem.

1.6 Methodology

The methodology followed in this project is iterative and incremental in nature. Instead of attempting to define and implement the entire system in a single pass, the project develops through a sequence of phases in which requirements are gathered, refined, implemented, and evaluated progressively. This approach is appropriate for user-oriented web applications because it allows design improvements to emerge through repeated review of usability, structure, and user flow.

The first stage involved identifying the practical difficulties faced by students during placement preparation. This was done through reflection on common student experiences, informal discussions, and observation of existing career-related platforms. Based on these observations, the initial requirements were identified and categorized into functional and non-functional groups. These requirements then became the basis for defining actors, use cases, and user interaction flows.

In the next phase, the conceptual design of the system was prepared. This included planning the homepage journey, identifying major components, selecting the technology stack, and deciding how authentication and onboarding should influence routing behavior. UML-oriented thinking was used in the form of use case, activity, and sequence modeling to describe the intended behavior of the system and to improve clarity before implementation.

Implementation proceeded in iterative modules. One set of iterations focused on building the visual structure of the homepage, including hero content, platform features, statistics, testimonials, and FAQ elements. Another set focused on routing logic and authentication integration through Clerk or a comparable framework. Additional refinement was then applied to user onboarding behavior, content arrangement, and overall responsiveness across devices.

Testing formed the final stage of the methodology. Rather than relying only on code-level assumptions, the project adopted practical validation through manual testing of user flows, routing scenarios, and content rendering. This methodology helped ensure that the resulting platform was not only conceptually sound but also operationally coherent from a user perspective.

1.7 Related Work and Existing Systems

The development of this project is informed by the broader landscape of educational technology, interview preparation systems, and career support platforms. Several categories of existing systems are relevant to this domain. These include general-purpose learning platforms, coding practice websites, corporate recruitment portals, community-driven interview experience websites, and modern AI-assisted educational tools. Each category contributes useful ideas, but none fully addresses the need for a guided and unified platform tailored to early career preparation in the way intended by the present project.

General learning platforms typically offer extensive course content and training resources, but they are often organized around subjects rather than around the specific journey of interview readiness. They may help users learn, but they do not always guide them through the practical workflow of placement preparation. Similarly, coding practice platforms are effective for technical exercises but usually concentrate on problem solving in isolation, leaving out broader elements such as onboarding, structured guidance, FAQs, testimonials, and a holistic user experience.

Corporate recruitment portals serve a different purpose altogether. They are designed primarily for company-specific application processes and candidate filtering, not for broad-based interview preparation. Community-driven sites that publish interview experiences or crowdsourced questions can be useful, but they are often inconsistent in quality and organization. They also lack the structured onboarding and integrated preparation flow that beginners often need.

Recent AI-powered tools have introduced new possibilities in educational and career technology by generating questions, explanations, and suggestions automatically. However, many such systems are general-purpose and not specifically tailored to the needs of final-year engineering students preparing for structured placement processes. The present project responds to this gap by combining UI clarity, onboarding flow, career-oriented messaging, and future AI extensibility into one targeted platform.

A simple conceptual comparison can be stated as follows:

- Generic learning platforms provide content depth but limited placement-flow guidance.
- Coding practice platforms provide technical practice but limited career-context integration.
- Recruitment portals provide application workflows but not preparation support.
- Community resources provide experience sharing but often lack structure and consistency.
- The proposed platform aims to combine structure, accessibility, trust elements, and future AI-oriented support within a unified career-preparation interface.

This comparison highlights the relevance of the current work. The contribution of the project lies not in replacing all other systems but in integrating the most useful aspects of structured preparation into a focused and coherent platform for student career readiness.

1.8 Organization of the Report

This report is organized into seven chapters followed by appendices, in accordance with the project structure described in the source document. Each chapter addresses a specific dimension of the project so that the complete development process can be understood in a logical sequence from problem identification to conclusion.

Chapter 1 introduces the background of the problem, the project motivation, the problem statement, objectives, scope, methodology, related work, and report organization. Chapter 2 presents the requirements and analysis of the system, including actors, functional requirements, non-functional requirements, requirement prioritization, and use case descriptions. Chapter 3 explains the software design, covering architecture, technology stack, component structure, data flow diagrams, activity diagrams, and sequence diagrams.

Chapter 4 focuses on database design, where data storage approach, entity identification, ER modeling, relational schema, and normalization principles are discussed. Chapter 5 presents the testing strategy, types of testing, test suites, detailed test cases, and the planned screenshot section for system validation. Chapter 6 documents the roles and responsibilities of the project team members, while Chapter 7 concludes the report by summarizing achievements, limitations, and future enhancement possibilities.

The appendices will include supplementary material such as extended test tables, glossary elements, placeholder references for screenshots, and other supporting artifacts as required for the final report format. This chapter-wise organization ensures that the report remains structured, academically consistent, and easy to evaluate.

CHAPTER 2

REQUIREMENTS AND ANALYSIS

The purpose of this chapter is to define in clear terms what the proposed system is expected to accomplish, who will interact with it, and what quality standards it must satisfy in order to be considered useful and reliable. Requirements analysis is a critical stage of software engineering because it transforms a broad project idea into a structured description of expected behavior, system boundaries, user roles, and implementation priorities. In the context of the present project, requirements analysis serves as the bridge between the conceptual goal of improving career guidance and the practical design of a web platform capable of supporting that goal.

For a project such as the AI-Powered Career Guidance and Interview Preparation Platform, requirements must be approached from both functional and non-functional perspectives. Functional requirements describe what the system should do, such as allowing users to register, navigate onboarding flows, view features, access FAQs, and reach the dashboard. Non-functional requirements define how well the system should perform these functions, such as being responsive, secure, scalable, and easy to maintain. Together, these requirements ensure that the system is not only complete in terms of features but also acceptable in terms of quality and usability.

This chapter also identifies the main actors interacting with the system, classifies project requirements according to priority, and describes representative use cases that capture important user journeys. By documenting these aspects clearly, the project establishes a formal basis for the software design, database structure, and testing activities discussed in later chapters.

2.1 User Types (Actors)

Any software platform becomes meaningful only when the intended users and their interactions are clearly defined. In this project, the user base is divided into a small set of actors that reflect the current scope of implementation while also leaving room for future expansion. These actors help determine not only system functionality but also interface design, routing behavior, access restrictions, and content visibility.

The primary actor of the system is the **Guest User**. A guest user is a visitor who accesses the platform without being authenticated. This user is allowed to browse the public-facing sections of the application, such as the landing page, features section, testimonials, and frequently asked questions. The guest user can understand what the platform offers, evaluate its usefulness, and decide whether to proceed further. However, because the guest user is not signed in, personalized experiences such as onboarding and dashboard access remain unavailable at this stage.

The second actor is the **Registered User**. This is a user who has successfully created an account and authenticated through the platform's sign-in mechanism. Once authenticated, the user is no

longer treated as a passive visitor but as an active participant in the platform. The registered user can move through onboarding, access protected routes, and eventually interact with more advanced platform features. In the context of the current implementation, this role is central because much of the routing logic depends on whether the user is signed in and whether onboarding has been completed.

A third actor, included as optional within the current scope, is the **Administrator**. While the present implementation is primarily focused on frontend experience and user navigation, the conceptual design anticipates an administrative role that can manage structured content such as interview questions, FAQs, testimonials, and general system configuration. Including the administrator at the requirements level is useful because it prepares the architecture for future content management needs without forcing the current version to implement a full admin dashboard.

These actors can be represented more formally as follows:

User Type	Description	Access Level	Main Purpose
Guest User	Visitor not signed into the platform	Public sections only	Explore platform and understand value proposition
Registered User	Authenticated user with account access	Public + protected user routes	Complete onboarding and access personalized features
Administrator (Optional)	Privileged content and configuration manager	Extended management privileges	Manage questions, FAQs, testimonials, and settings

The actor model used here is intentionally simple, which is appropriate for the current phase of the project. A more advanced version of the system could introduce additional actors such as mentors, recruiters, career coaches, or institutional coordinators. However, for the present report, the defined actor set is sufficient to explain the implemented workflows and proposed system behavior.

2.2 Functional Requirements

Functional requirements specify the services, interactions, and behaviors that the platform must support in order to fulfill its objective. Since the proposed system is designed as a guided career preparation environment, the requirements are centered on user access control, onboarding progression, content presentation, and route-based personalization. These functions collectively define the visible and interactive behavior of the platform.

The first major functional requirement is **user registration and authentication**. The system must allow new users to sign up and existing users to sign in using a secure authentication provider. It must preserve signed-in state appropriately across protected routes and ensure that unauthorized users cannot directly access restricted areas such as onboarding or dashboard sections. This

requirement is foundational because all personalized behavior depends on secure and reliable user identity handling.

The second functional requirement is the **onboarding flow**. After registration, the system should direct new users through an onboarding process that collects basic information such as educational background, interests, and career goals. The platform should maintain a logical flag or state indicator showing whether onboarding has been completed. This allows the system to make routing decisions dynamically and ensures that users experience the platform in a structured progression rather than jumping into deeper features without setup.

The third major requirement concerns the **landing page and call-to-action flow**. The homepage must communicate the platform's value clearly and include a strong "Get Started" or equivalent call-to-action button. When the user clicks this action, the system must determine whether the user is unauthenticated, authenticated but not onboarded, or fully ready for dashboard access, and it must redirect accordingly. This routing behavior transforms the landing page from a static visual page into an active decision gateway.

Another important requirement is the **features section**, which must display a set of platform capabilities in a clear and structured format. Each feature should appear with an icon, title, and short explanation so that users can quickly understand the benefits of the system. This section has informational as well as persuasive value, since it helps convert a visitor's curiosity into trust and engagement.

The **statistics section** is also functionally relevant because it provides users with visible confidence indicators such as number of industries covered, size of content library, success rate indicators, or AI support presence. Although such metrics may appear purely visual, they play an important role in communicating scale and credibility. Therefore, the platform should present them clearly and consistently.

The **"How It Works" section** functions as a guided explanation of the user journey. It must explain the key steps involved in using the platform, such as account creation, preference setup, practice flow, and feedback interaction. This requirement is essential because one of the project's main goals is to reduce ambiguity and provide structured progression instead of leaving the user to interpret the system independently.

The **testimonials section** provides social proof and helps establish trust in the platform. The system must be capable of displaying testimonial entries with names, roles, organizations, images, and short statements. The layout must be readable and visually consistent so that the section contributes to credibility rather than clutter.

Similarly, the **FAQ section** must allow users to view common questions and expand answers as needed. This should ideally be implemented through an accordion-style component so that content remains organized while still being accessible. FAQs reduce friction by answering common concerns about the platform without requiring direct support.

Finally, once the registered user has completed onboarding, the system must provide **dashboard access**. The dashboard is intended to act as the entry point to deeper personalized experiences and future AI-enabled modules. Even if the current dashboard implementation is basic, the routing structure must treat it as a protected destination reached only after appropriate user progression.

A consolidated view of major functional requirements is shown below:

Req. ID	Functional Requirement	Description
FR1	User Registration and Authentication	Allow secure sign-up, sign-in, and protected route management
FR2	Onboarding Flow	Collect user profile information and maintain onboarding state
FR3	Landing Page CTA Routing	Redirect users based on authentication and onboarding status
FR4	Features Section	Present platform features with icons, titles, and descriptions
FR5	Statistics Section	Display confidence-building numerical platform metrics
FR6	How It Works Section	Show step-by-step preparation workflow
FR7	Testimonials Section	Display structured user testimonials and trust elements
FR8	FAQ Section	Provide expandable answers to common user questions
FR9	Dashboard Access	Allow onboarded users to reach protected personalized area

These functional requirements define the minimum working behavior of the system in its present form. Together, they ensure that the platform is informative, navigable, secure, and extensible enough to support future enhancements without losing structural clarity.

2.3 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational expectations of the system. While functional requirements describe what the system does, non-functional requirements determine how effectively and reliably those functions are delivered. For a platform intended to guide students through career preparation, quality is especially important because poor usability, weak performance, or confusing design could discourage users even if the core functionality is technically present.

The first and perhaps most visible non-functional requirement is **usability**. The system should present an intuitive and consistent interface that users can understand without extensive

instruction. Layout, typography, spacing, icons, and navigation should work together to create a simple and welcoming experience across multiple devices. Since the platform is intended for students who may access it through laptops, tablets, or phones, responsiveness is a critical part of usability rather than a secondary enhancement.

The second requirement is **performance**. The platform should load pages quickly, navigate smoothly, and minimize unnecessary delay in visual rendering and route transitions. Static sections such as features, testimonials, and FAQs should appear efficiently, and authentication-based routing should not create confusing lag or repeated reload behavior. Good performance contributes directly to user trust and platform professionalism.

The third major quality requirement is **security**. Because the project involves user registration and authenticated access, sensitive user state and identity data must be handled through secure mechanisms. Authentication and authorization should rely on established frameworks, and protected routes must not expose user-specific sections to unauthorized access. Even in an academic project, security remains a core design consideration because insecure systems fail to inspire confidence.

Another important quality attribute is **scalability**. Although the current project is designed as a foundational system, its architecture should be capable of supporting increasing numbers of users, content entries, and future modules. The use of modern frameworks such as Next.js contributes to deployment flexibility and scalable application structure. Scalability is particularly relevant here because the platform is intended to support future extensions like AI modules, analytics, and expanded question banks.

The requirement of **maintainability** is equally important. The codebase should be modular, readable, and structured in a way that makes future edits manageable. Naming conventions, component organization, styling consistency, and logical separation of responsibilities all contribute to maintainability. Since the project is both an academic submission and a potential foundation for future development, clean structure is necessary not only for present success but for long-term viability.

Finally, the system should demonstrate **reliability**. It should handle common issues such as failed navigation states, temporary loading conditions, or partial content rendering gracefully. Users should receive meaningful behavior even when ideal conditions are not available. Reliability does not imply enterprise-level fault tolerance in the current scope, but it does require a stable and predictable experience under typical conditions.

A summary of non-functional requirements is given below:

Req. ID	Non-Functional Requirement	Description
NFR1	Usability	Intuitive, consistent, and responsive user interface
NFR2	Performance	Fast loading, efficient rendering, and smooth navigation

Req. ID	Non-Functional Requirement	Description
NFR3	Security	Safe authentication, authorization, and route protection
NFR4	Scalability	Ability to support future growth in users and features
NFR5	Maintainability	Modular, readable, and extensible code structure
NFR6	Reliability	Stable behavior under normal use and minor disruptions

These quality requirements ensure that the platform is not merely functional in a narrow sense but also practically usable, academically credible, and technically sustainable.

2.4 Requirement Classification (MoSCoW)

When designing a software system with limited time, effort, and implementation scope, it becomes necessary to distinguish between essential requirements and desirable extensions. The MoSCoW prioritization method provides a useful framework for this purpose by classifying system requirements into four categories: **Must-have**, **Should-have**, **Could-have**, and **Won't-have for now**. Applying this method allows the project team to focus implementation on high-value features while still documenting the broader product vision.

The **Must-have** category includes requirements without which the platform would fail to achieve its primary purpose. In the present project, must-have elements include authentication, onboarding-state logic, hero-section call-to-action routing, feature presentation, testimonials, FAQs, and protected dashboard routing. These capabilities collectively define the minimum meaningful version of the system.

The **Should-have** category contains features that significantly improve user experience but are not strictly required for core functionality. Structured statistics, high-quality "How It Works" guidance, responsive multi-device presentation, and basic AI placeholders fall into this category. These items elevate the platform's professionalism and clarity while remaining secondary to the platform's core access and navigation logic.

The **Could-have** category includes features that are valuable but not necessary for the present stage of delivery. Examples include role-specific interview preparation tracks, deeper user analytics, or an administrative panel for content management. These ideas are aligned with the platform's future potential but are intentionally deferred to maintain focus in the current version.

The **Won't-have for now** category identifies features that are explicitly outside current scope. This includes payment integration, large-scale recommendation engines, and a dedicated full mobile application. Defining these exclusions is important because it protects the project from uncontrolled scope expansion and helps evaluators understand the deliberate boundaries of implementation.

The classification can be summarized in table form:

Priority	Included Items
Must-have	Authentication, onboarding logic, CTA routing, features, testimonials, FAQs, basic dashboard routing
Should-have	Statistics section, “How It Works” flow, responsive design, AI placeholders
Could-have	Role-specific tracks, deeper analytics, admin content panel
Won’t-have for now	Payment system, large recommendation engine, full mobile app

This prioritization reflects practical engineering judgment. It shows that the project is focused on delivering a stable, coherent, and demonstrable system first, while still acknowledging the larger roadmap for future development.

2.5 Use Case Descriptions

Use cases describe the interactions between users and the system in goal-oriented terms. They help translate abstract requirements into real user situations and clarify what happens when a user attempts a particular action. In this project, use cases are especially important because the platform is built around guided progression, and much of its value lies in how the system responds to user state, intent, and navigation path.

UC1: Register and Log In represents the initial access path for new users. In this use case, the actor is the guest user, whose precondition is that no authenticated session currently exists. The main flow begins when the user selects a sign-up or sign-in option and submits credentials through the authentication provider. On successful authentication, the user is redirected back into the application context. This use case establishes the identity foundation on which all protected interactions depend.

UC2: Get Started from Home Page is one of the most important platform flows. The actor may be either a guest user or a registered user. The user clicks the “Get Started” button on the home page, after which the system checks authentication and onboarding status. If the user is not signed in, the route shifts toward sign-up. If signed in but not onboarded, the route moves to onboarding. If both conditions are satisfied, the user is directed to the dashboard. This use case captures the project’s central routing intelligence.

UC3: View Features applies to both guest and registered users. The main flow involves the user navigating to the features section and reading the described platform benefits. An alternate flow may include hovering over cards or tapping them on smaller devices to understand emphasis or interaction states. Though simple, this use case contributes directly to value communication and user conversion.

UC4: View “How It Works” Steps describes the interaction where the user reads the platform’s guided workflow. This use case is important because one of the project’s objectives is to reduce

uncertainty and make the preparation process understandable in sequential form. The user scrolls to the section and observes the key steps in the intended usage journey.

UC5: View Testimonials captures the trust-building behavior where a user reads other users' statements, roles, and company associations. This use case may appear passive, but it has persuasive significance because it influences perceived credibility and platform legitimacy.

UC6: Expand FAQ describes the interaction with the accordion-based frequently asked questions area. The user selects one question, the system expands the answer, and the user reads the content. The alternate path occurs when the user collapses the current item and opens another. This use case reduces information friction and supports self-service clarification.

UC7: Access Dashboard (Post-Onboarding) applies to registered users who have completed the onboarding phase. The system validates user status and routes the user to the dashboard, which serves as a personalized entry point to deeper features and future modules. This use case represents successful progression through the public and onboarding layers of the platform.

A concise summary of these use cases is shown below:

Use Case ID	Use Case Name	Primary Actor	Outcome
UC1	Register and Log In	Guest User	User authenticates successfully
UC2	Get Started from Home Page	Guest / Registered User	Routed to sign-up, onboarding, or dashboard
UC3	View Features	Guest / Registered User	User understands platform capabilities
UC4	View How It Works	Guest / Registered User	User understands process steps
UC5	View Testimonials	Guest / Registered User	User gains confidence through social proof
UC6	Expand FAQ	Guest / Registered User	User views answers to common questions
UC7	Access Dashboard	Registered User	User enters protected personalized area

These use cases make the system's behavior concrete and provide a clear foundation for later diagrammatic representation and testing design.

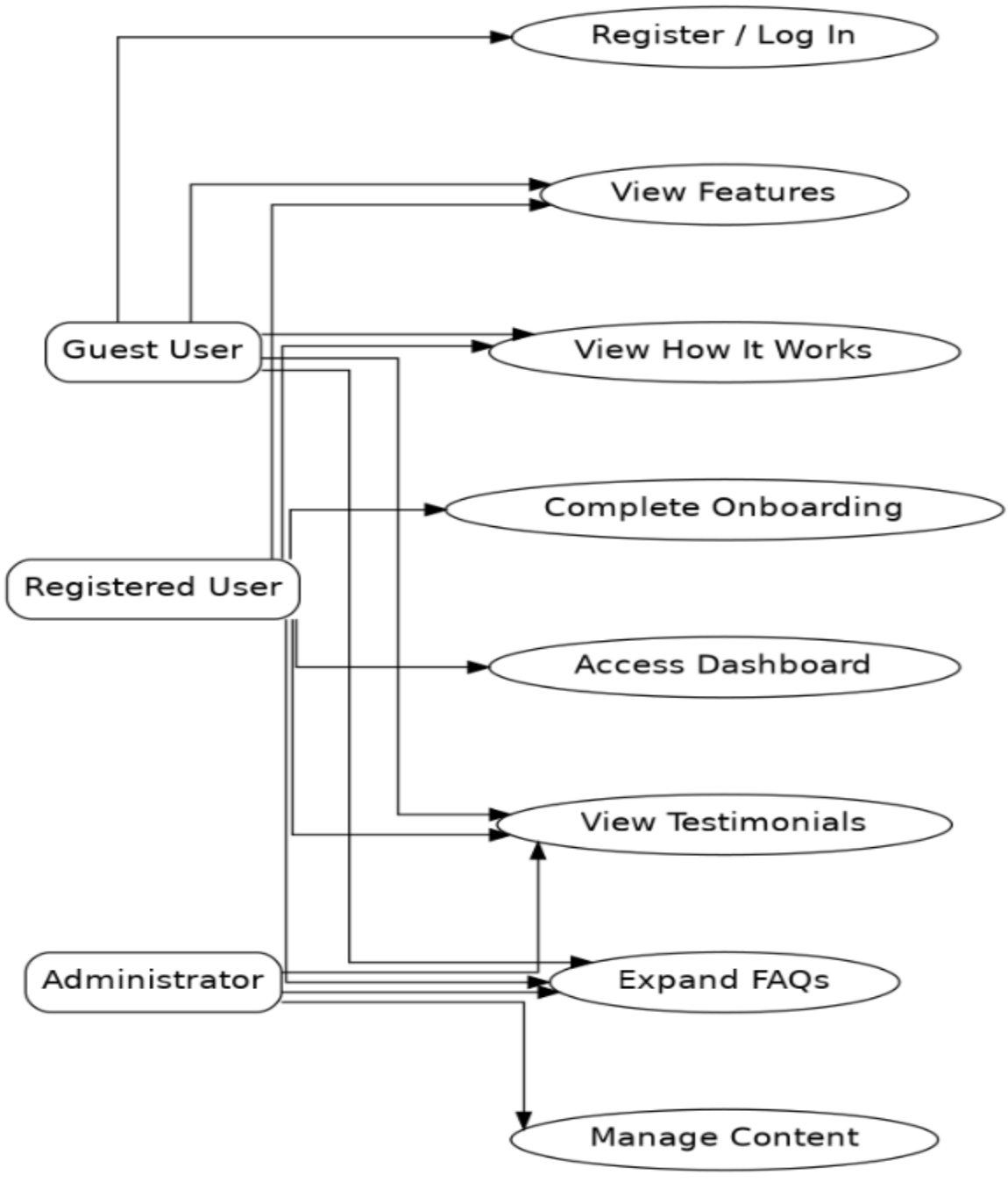
2.6 Use Case Diagram (Narrative Description)

The use case diagram for the proposed system can be described narratively by examining how each actor relates to the platform's key interactions. The **Guest User** interacts primarily with

public-facing actions such as registering, viewing platform features, reading the “How It Works” section, exploring testimonials, and expanding FAQs. These actions support awareness, trust-building, and initial conversion into a registered user.

The **Registered User** inherits the public interactions available to the guest user but also gains access to additional actions such as completing onboarding and reaching the dashboard. This distinction is important because the platform’s value increases after authentication, and the use case model must reflect that progression rather than treating all users as equivalent.

If the optional **Administrator** role is included in a future version, that actor would be associated with management-oriented use cases such as maintaining content libraries, updating FAQ entries, managing testimonials, and configuring broader platform behavior. Even if such flows are not fully implemented now, representing them conceptually helps align future system evolution with current design thinking.



CHAPTER 3

SOFTWARE DESIGN

Software design is the stage in which identified requirements are translated into a structured technical solution. While requirements analysis explains what the system should do, software design explains how those expectations are realized through architecture, technology choices, component structure, control flow, and interaction logic. For the AI-Powered Career Guidance and Interview Preparation Platform, software design is particularly important because the project aims to provide a smooth and guided user experience rather than a collection of disconnected pages. The design must therefore support clarity, modularity, controlled navigation, and future extensibility.

The present project is built as a modern web application using technologies that support component-based development, responsive user interface design, authentication integration, and scalable routing. The design places strong emphasis on separation of concerns. User interface components are responsible for presentation and interaction, authentication services handle identity-related flows, routing logic governs navigation decisions, and data structures support future persistence and personalization. This layered arrangement improves maintainability and makes the system easier to extend over time.

The software design of the system may be understood at multiple levels. At the highest level, the platform follows a layered architecture consisting of client, application, and data layers. At a more detailed level, it is composed of reusable interface components such as the hero section, features section, statistics section, testimonials section, and FAQ accordion. At the process level, user interactions can be represented through data flow diagrams, activity diagrams, and sequence diagrams. Together, these views provide a complete picture of the system from both structural and behavioral perspectives.

3.1 System Architecture

The system follows a layered web application architecture in which responsibilities are distributed across distinct but cooperating layers. This design is appropriate for user-facing web platforms because it keeps presentation logic, routing logic, and data-related behavior conceptually separated while still allowing efficient communication among them. In the current project, the architecture can be broadly described in terms of a **client layer**, an **application layer**, and a **data layer**.

The **client layer** represents the frontend of the platform, implemented using Next.js and React and rendered within the user's browser. This layer contains all user-visible sections of the application, including the landing page, feature descriptions, statistics, "How It Works" workflow, testimonials, FAQs, and final call-to-action areas. It is responsible for layout, visual consistency, interaction controls, and presentation of structured content. Because the platform is designed to guide users progressively, the client layer must also support clear content hierarchy and intuitive navigation cues.

The **application layer** contains routing logic, server-side support capabilities of the framework, and authentication-related integration. This is the layer where decisions are made based on whether the user is signed in, whether onboarding has been completed, and which route should be shown next. It acts as the coordinator of the platform's user journey. When a user clicks "Get Started," for example, the application layer interprets user state and determines whether the next page should be sign-up, onboarding, or dashboard access.

The **data layer** represents persistent or semi-persistent storage used to maintain user profiles, onboarding information, interview questions, testimonials, and frequently asked questions. Even if the current implementation focuses more heavily on frontend and access flow, the architectural design anticipates a proper data layer so that the project remains realistic and extensible. This layer is essential for storing user-specific information and for enabling future AI-supported features such as personalization, recommendation, and progress tracking.

Communication between these layers occurs primarily through secure web protocols and internal framework mechanisms. The client interacts with the application layer through route navigation, event handling, and possible API or server-side calls. The application layer, in turn, interacts with the data layer through database queries or abstracted access methods. Static resources such as testimonial images and visual assets are delivered through optimized frontend asset handling pipelines. This creates an architecture that is modular enough for future growth while remaining manageable within the academic scope of the present project.

A textual representation of the architecture is shown below:

Layer	Main Responsibilities
Client Layer	User interface rendering, section display, interactions, responsive design
Application Layer	Routing logic, authentication flow, onboarding checks, server-side coordination
Data Layer	User profile storage, content records, question banks, testimonials, FAQ persistence

This architecture supports the project's central aim of providing a guided and personalized user journey. It also ensures that the system is not tightly coupled to a single page or a single use case, but instead capable of being developed in a modular and extensible fashion.

3.2 Technology Stack

The technology stack selected for the project reflects a balance between developer productivity, performance, maintainability, and suitability for modern web application development. Since the project emphasizes user experience, modularity, and scalability, the chosen tools support rapid interface construction while also offering structure for more advanced future features.

The primary framework used is **Next.js**, which provides a React-based architecture for building web applications with support for file-based routing, server-side rendering, static site generation,

and optimized application performance. This framework is especially useful for academic and practical projects because it reduces boilerplate complexity while offering production-style structure. It is well suited for systems in which navigation, page organization, and fast rendering are important.

The user interface is built using **React**, a library that encourages component-based development. React allows the application to be broken into reusable sections such as hero banners, cards, accordions, and statistics displays. This improves readability, reduces repetition, and makes future changes easier. In the context of the current project, React is valuable because many sections of the homepage are modular by nature and benefit from reusable component design.

For authentication and user management, the project uses **Clerk or a similar authentication service**. This helps simplify secure sign-in, sign-up, session management, and access control. Instead of implementing authentication from scratch, which could introduce unnecessary complexity and security risk, the project relies on an established solution that integrates with the application's routing logic and helps manage authenticated user state more safely and efficiently.

The project also benefits from **TypeScript**, either as an active implementation tool or as a strongly recommended enhancement. TypeScript extends JavaScript with static typing, which reduces runtime ambiguity and improves code clarity, especially as the system grows. Though not mandatory for a minimal implementation, it significantly strengthens maintainability and development discipline.

For visual styling and layout consistency, **Tailwind CSS and component libraries** are used or recommended. These tools allow efficient and scalable styling through utility classes and reusable visual conventions. This is highly suitable for a project that depends on card layouts, responsive spacing, section-wise visual separation, and modern aesthetic presentation.

Finally, the data layer may be supported by a database system such as **PostgreSQL**, **MySQL**, or **MongoDB**, depending on implementation preference and deployment strategy. In the conceptual design of this report, the database is treated as an essential long-term component even if the current focus is more on frontend behavior and structured content flow.

A concise technology summary is shown below:

Technology	Role in the System
Next.js	Web framework, routing, rendering, project structure
React	Reusable UI component development
Clerk / Similar	Authentication, session management, access control
TypeScript	Static typing and improved code robustness
Tailwind CSS / UI Libraries	Styling, responsiveness, modern interface design

Technology	Role in the System
PostgreSQL / MySQL / MongoDB	Persistent storage for users and content

The selected stack is appropriate for an academic major project because it demonstrates contemporary industry tools while remaining realistic for student development. It also leaves the project well positioned for future expansion into AI-assisted modules and deeper data management.

3.3 Component Design

Component design focuses on how the user interface is divided into smaller, reusable, and logically organized units. Since the project is a web-based platform with multiple visible sections, a component-oriented approach improves maintainability and simplifies future extension. Each component is designed to represent a specific functional or informational segment of the homepage and related flows.

The **Hero Section Component** is the first major UI element. It displays the platform's title, supporting description, and the primary call-to-action button such as "Get Started." This component is critical because it forms the entry point to the user journey and initiates routing logic. It is not merely decorative; it is functionally tied to authentication and onboarding checks through event handling.

The **Features Section Component** presents the main capabilities of the platform in a card-based layout. Each feature card typically contains an icon, title, and descriptive summary. The purpose of this component is to communicate value quickly and visually, allowing users to understand what the platform offers without reading long paragraphs. Structuring this data-driven section as a component makes it easier to update features later.

The **Statistics Section Component** is designed to highlight confidence-building numbers such as industries covered, number of interview questions, success-related metrics, or AI support claims. This component strengthens user trust and provides a visual pause between explanatory sections. Because the content is numerical and visually repetitive in structure, a component approach is especially effective.

The **How It Works Section Component** explains the platform's workflow in sequential form. It helps users understand the intended process from account creation to guided preparation and dashboard access. This section is central to the platform's identity because the project aims to reduce confusion through structured progression. The component therefore acts as an instructional interface element rather than just an informational one.

The **Testimonials Section Component** renders user reviews or social proof elements in a card format. These cards may include user image, name, role, company, and a short quote. Since trust is a major factor in platform adoption, the testimonial component contributes directly to user confidence and perceived legitimacy.

The **FAQ Section Component** is implemented using an accordion-like interaction model in which questions are expanded and collapsed. This is an efficient design pattern because it allows multiple questions to be displayed without overwhelming the page. It also encourages user-driven exploration of common doubts and removes friction in decision-making.

The **Call-to-Action Footer Component** appears near the end of the page and serves as a second opportunity for user conversion. After moving through features, workflow explanation, and trust-building sections, the user encounters a final action prompt that reinforces the next step. This supports the idea that the homepage is a carefully guided narrative rather than a random sequence of content blocks.

A simple summary of core frontend components is given below:

Component	Purpose
Hero Section	Entry point, value proposition, CTA trigger
Features Section	Communicate key platform capabilities
Statistics Section	Build credibility through visual metrics
How It Works	Explain preparation flow step by step
Testimonials	Provide social proof and trust
FAQ	Answer common user questions efficiently
CTA Footer	Encourage final action after full page journey

The use of modular components makes the project easier to manage, easier to document, and more compatible with future enhancement. It also aligns with best practices in modern frontend engineering.

3.4 Data Flow Diagrams (DFD)

Data Flow Diagrams are used to show how information moves through the system, how external entities interact with it, and how internal processes transform and route data. In the current project, DFDs are especially useful because the platform's behavior depends on user actions, authentication responses, content retrieval, and navigation state transitions.

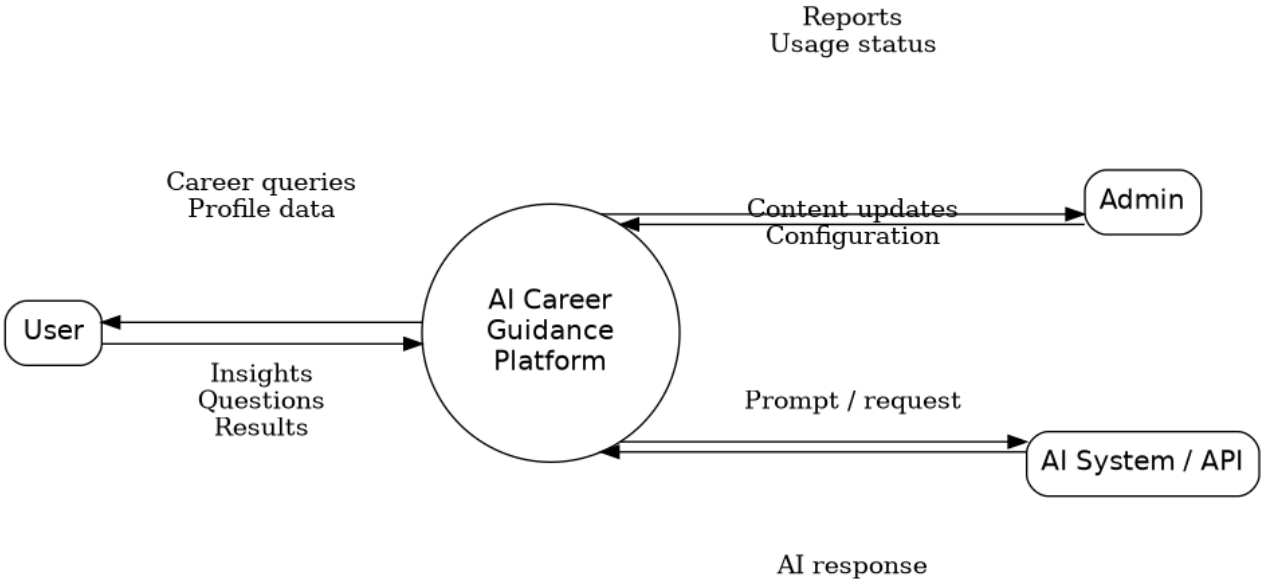
At **Level 0**, also known as the context diagram, the entire platform is represented as a single high-level process interacting with external entities. The main external entity is the **User**, who provides inputs such as clicks, registration details, onboarding information, and content interaction signals. Another entity is the **Authentication Provider**, which exchanges sign-in and session-related information with the platform. An optional **Administrator** may also interact with the system in future versions by updating content such as testimonials, FAQs, and interview questions.

The Level 0 diagram helps establish the boundaries of the platform. It shows that the AI-Powered Career Guidance Platform acts as a central process receiving user requests, requesting or

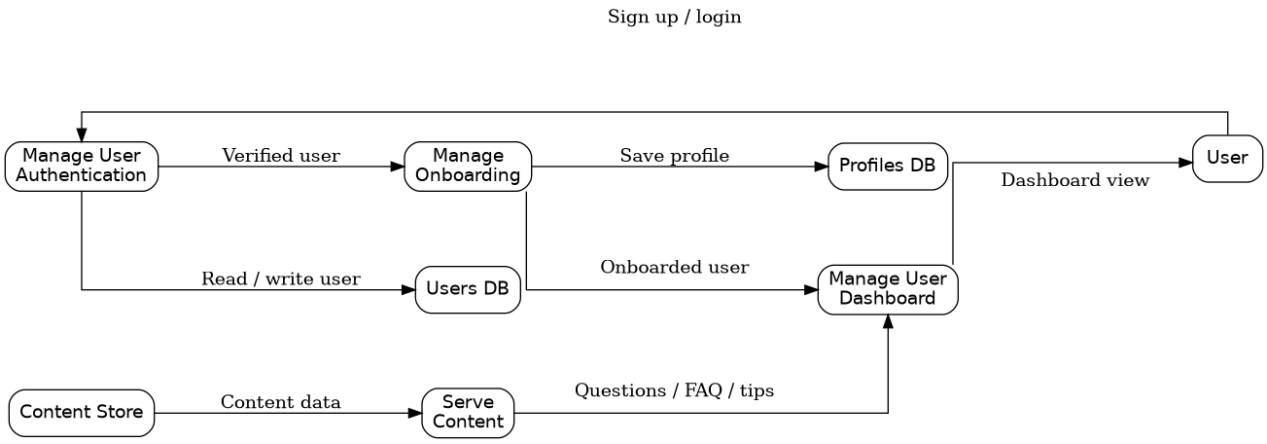
validating authentication, and returning content or route outcomes. In more advanced interpretations, the platform may also exchange data with AI systems, analytics modules, or resume-processing APIs, but such interactions remain conceptual in the current phase.

At **Level 1**, the single system process is decomposed into smaller internal processes such as **Manage User Authentication**, **Manage Onboarding**, **Serve Content**, and **Manage User Dashboard**. This provides a more realistic view of internal system behavior. Authentication management communicates with the auth provider and user session data. Onboarding management reads and writes user profile details. Content serving retrieves structured platform content such as features, FAQs, and testimonials. Dashboard management handles personalized route delivery for onboarded users.

The value of these DFDs lies in their ability to show how a seemingly simple homepage is actually part of a structured system of checks, decisions, and data movements. For the final report, full diagram figures can be inserted later at the relevant points. In the present draft, it is sufficient to document the logic and reserve figure placeholders for the graphical representations.



[Fig. 3.1: Level 0 Data Flow Diagram of AI Career Guidance Platform]



[Fig. 3.2: Level 1 Data Flow Diagram of AI Career Guidance Platform]

3.5 Activity Diagrams

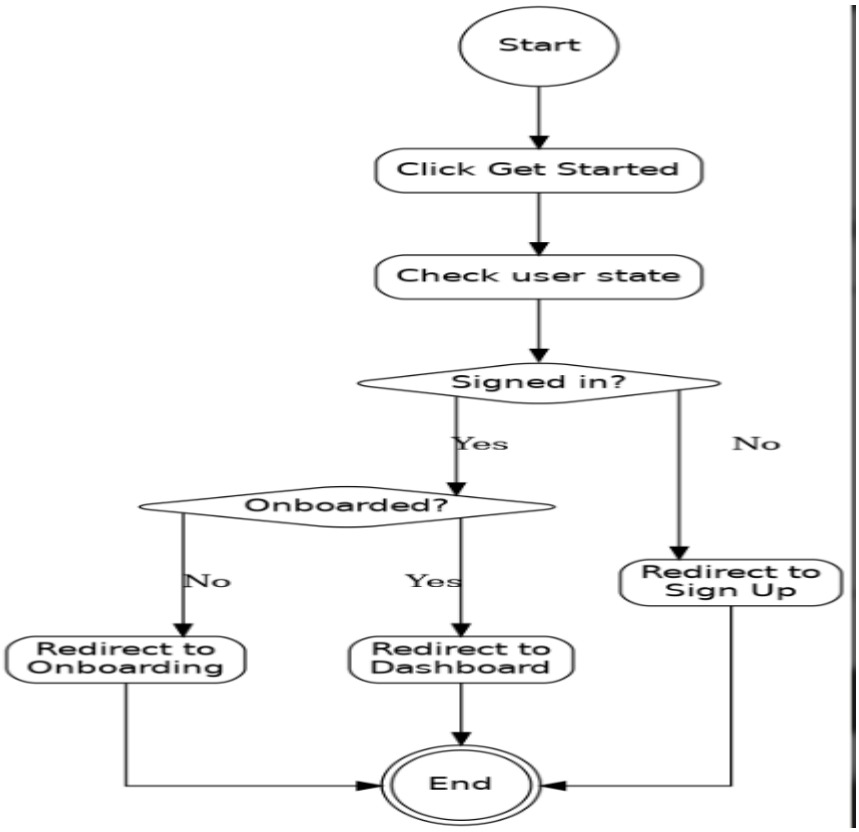
Activity diagrams are used to describe the flow of control through user interactions and system decisions. They are particularly useful in systems where route transitions depend on conditions such as authentication status, onboarding completion, or user-triggered expansion behavior. For this project, activity diagrams help explain how the platform guides users from initial interaction to subsequent outcomes.

One important activity flow is the **Get Started workflow**. The process begins when the user clicks the "Get Started" button on the homepage. The system first checks whether the required user data is loaded. If not, it waits until the state becomes available. Once data is ready, the system evaluates whether the user is signed in. If the user is not signed in, the flow redirects to the sign-up page. If the user is signed in but not onboarded, the system redirects to the onboarding flow. If both conditions are satisfied, the user is routed to the dashboard. This activity captures one of the most important dynamic behaviors of the platform.

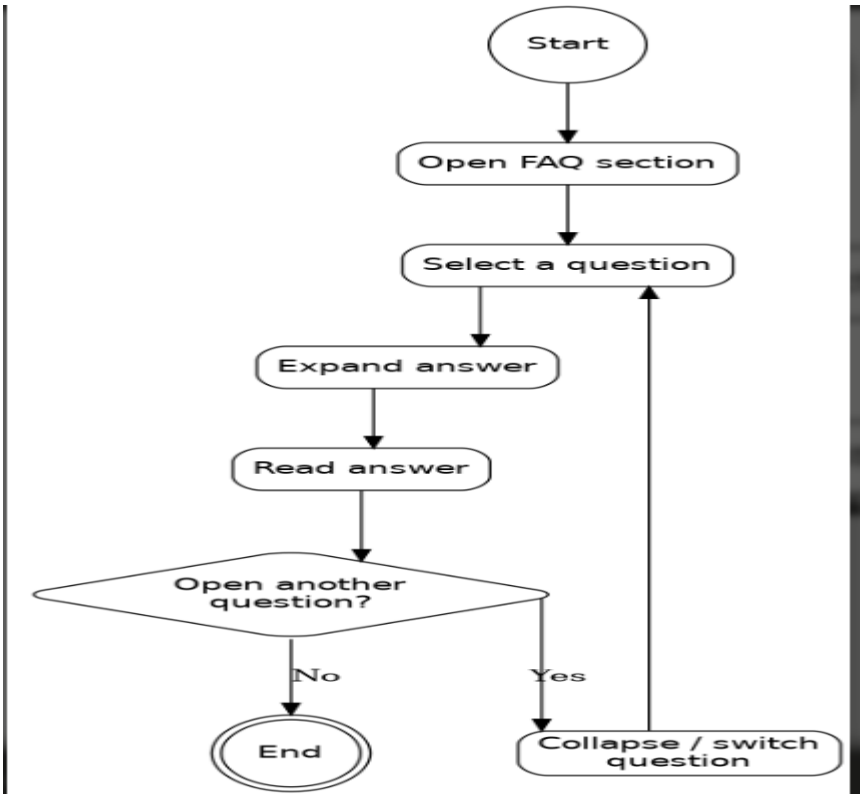
Another meaningful activity flow is the **Expand FAQ interaction**. In this case, the user navigates to the FAQ section, selects a question, and the system expands the answer using accordion logic. The user then reads the response and may either collapse the current question or open another one. This activity is simple, but it reflects the responsive and interactive nature of the interface.

A third useful activity flow is the **View Testimonials process**, in which the user scrolls to the testimonials section, the system renders testimonial cards, and the user reads the content before continuing further down the page. This interaction contributes to trust-building and page engagement, even though it is less decision-heavy than authentication-related flows.

Additional activity diagrams may later be prepared for registration, onboarding completion, dashboard access, or AI-assisted advice flow if those behaviors are documented in greater depth. For the current report structure, however, the most essential activities are sufficient to illustrate the project's dynamic behavior.



[Fig. 3.3: Activity Diagram for Get Started Flow]



[Fig. 3.4: Activity Diagram for FAQ Interaction]

3.6 Sequence Diagrams

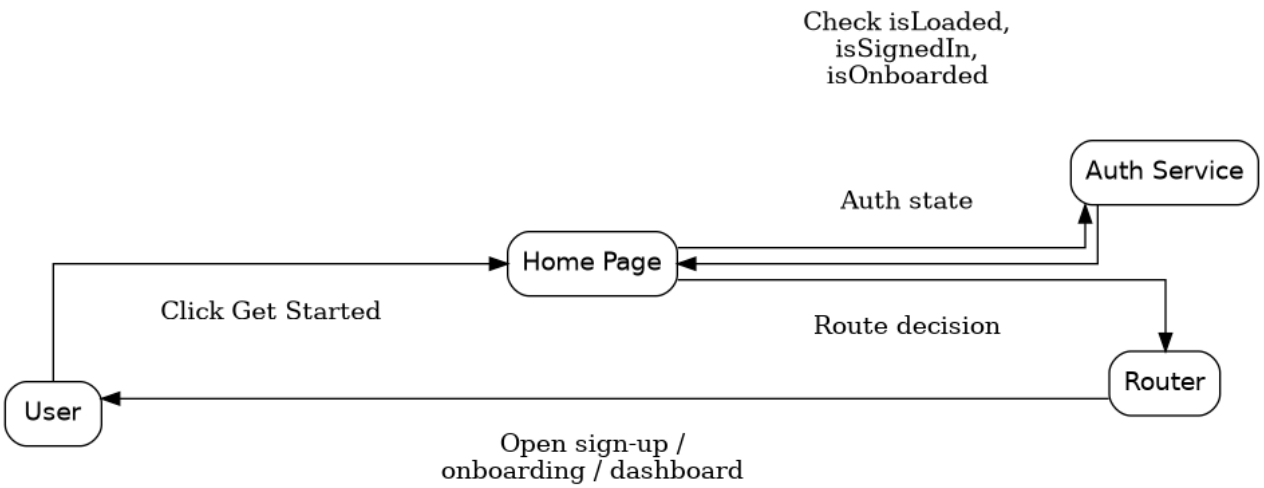
Sequence diagrams capture interactions between actors and system components over time. Unlike DFDs, which emphasize data movement, sequence diagrams emphasize the chronological order in which actions and responses occur. In the present project, sequence diagrams are useful because several user interactions depend on a clear chain of requests, checks, and UI updates.

One core sequence is the **Get Started button interaction**. The sequence begins when the user clicks the button on the homepage. The homepage component invokes a handler such as handleGetStarted, which then consults the authentication service for values like loading state, sign-in status, and onboarding status. Based on these conditions, the routing component directs the user to the appropriate destination such as sign-up, onboarding, or dashboard. This sequence clearly expresses how UI interaction, authentication logic, and route navigation cooperate in a time-ordered process.

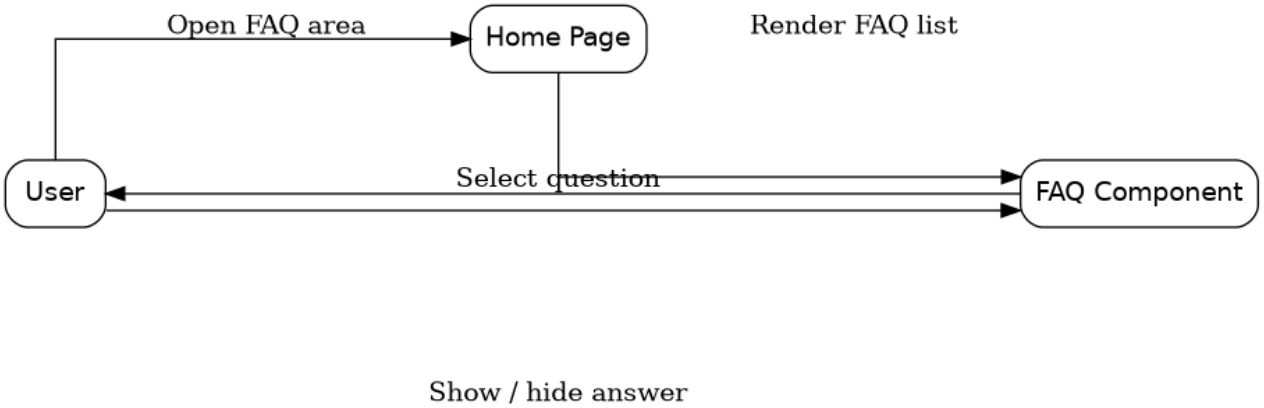
A second useful sequence is the **Load Testimonials interaction**. Here, the user scrolls to the relevant section, the homepage renders the testimonials component with structured data, and the component outputs the testimonial cards in the interface. While simple, this sequence shows the relationship between scroll-based user presence, component rendering, and content display.

A third sequence is the **Expand FAQ interaction**, where the user clicks a question item, the FAQ component toggles its internal state, and the answer content becomes visible or hidden accordingly. This sequence captures the interactive nature of the accordion design and demonstrates how small UI components still follow meaningful interaction lifecycles.

These sequence diagrams strengthen the documentation by showing that the project has been considered not only in terms of static layout but also in terms of ordered interaction logic. For the final report, graphical UML diagrams can be added later at the indicated locations.



[Fig. 3.5: Sequence Diagram for Get Started and Routing]



[Fig. 3.6: Sequence Diagram for FAQ Expansion]

CHAPTER 4

DATABASE DESIGN

Database design provides the structural foundation for storing, organizing, and retrieving the information required by the application. Even though the current stage of the project is strongly focused on frontend presentation and navigation flow, a realistic and scalable implementation requires a proper backend data model. Without a well-planned database structure, user onboarding data, interview question libraries, testimonials, and frequently asked questions would either remain static or become difficult to manage and extend. Therefore, database design is an important part of making the platform academically complete and technically future-ready.

The purpose of this chapter is to describe how the system's persistent information can be modeled in an organized and normalized manner. It includes the choice of storage approach, identification of main entities, the relationships among those entities, a proposed relational schema, and design considerations such as normalization and extensibility. Although the present implementation may not expose all backend capabilities in full production depth, this design establishes a formal data foundation for future deployment.

4.1 Data Storage Approach

The platform can be supported by either a relational or a document-oriented database, depending on developer familiarity, hosting constraints, and the complexity of the future feature set. However, for the purpose of academic clarity and structured data modeling, a **relational database approach** is the most suitable choice for this project. Relational databases provide clear schema definition, support for integrity constraints, and straightforward representation of entities such as users, profiles, industries, interview questions, testimonials, and FAQs.

A relational system such as PostgreSQL or MySQL allows data to be organized into tables with keys and relationships. This model aligns well with the kind of information handled by the platform. For example, each user can be associated with one profile, each interview question can be linked to a particular industry, and content entities such as FAQs and testimonials can be managed independently. Such a structure improves consistency and makes the system easier to query, test, and scale.

From an academic perspective, relational modeling is also beneficial because it allows the project to demonstrate important database concepts such as primary keys, foreign keys, normalization, and schema-level separation of concerns. These aspects strengthen the quality of the final report and show that the project has been considered beyond surface-level interface design.

4.2 Entity Identification

The first step in database design is identifying the major entities that represent real objects or conceptual data groups in the system. In this project, the core entities are derived directly from the platform's intended functionality and content organization.

The most fundamental entity is **User**. This entity represents a registered account in the platform and stores basic identity-related information. Typical attributes include user_id, name, email, auth_provider_id, role, and onboarded_flag. The onboarded_flag is especially important because it allows the system to determine whether the user should be routed to onboarding or to the dashboard after authentication.

The second important entity is **Profile**. While the User entity stores basic identity and access metadata, the Profile entity stores richer educational and preference information related to the user's career context. Attributes may include profile_id, user_id, education, skills, preferred_industry, and experience_level. Separating profile data from user identity data improves flexibility and supports cleaner normalization.

Another key entity is **Industry**, which acts as a categorization table for industry domains or preparation tracks. This entity may include attributes such as industry_id and industry_name. By using a dedicated industry table, the system avoids repeatedly storing the same textual industry labels for multiple questions or records.

The **InterviewQuestion** entity stores structured interview preparation content. It may include fields such as question_id, industry_id, question_text, difficulty_level, and sample_answer_hint. This entity is important because it supports one of the platform's core content goals: helping users prepare through organized and categorized question resources.

The **Testimonial** entity stores social proof records shown in the testimonials section. Its attributes may include testimonial_id, author_name, role, company, image_path, and quote. Keeping testimonials in a dedicated table allows easier content maintenance and possible future admin control.

The **FAQ** entity stores frequently asked questions and their corresponding answers. Relevant attributes include faq_id, question_text, and answer_text. This entity supports the FAQ accordion section and keeps support-oriented content separate from career-question content.

These entities can be summarized as follows:

Entity	Purpose
User	Stores registered account and onboarding state
Profile	Stores educational and preference details of user
Industry	Stores industry categories for content organization
InterviewQuestion	Stores categorized interview preparation questions
Testimonial	Stores user testimonial content
FAQ	Stores common questions and answers

Together, these entities capture the minimum persistent structure necessary for the platform to operate as a guided, extensible, and content-driven web system.

4.3 Entity–Relationship (ER) Model

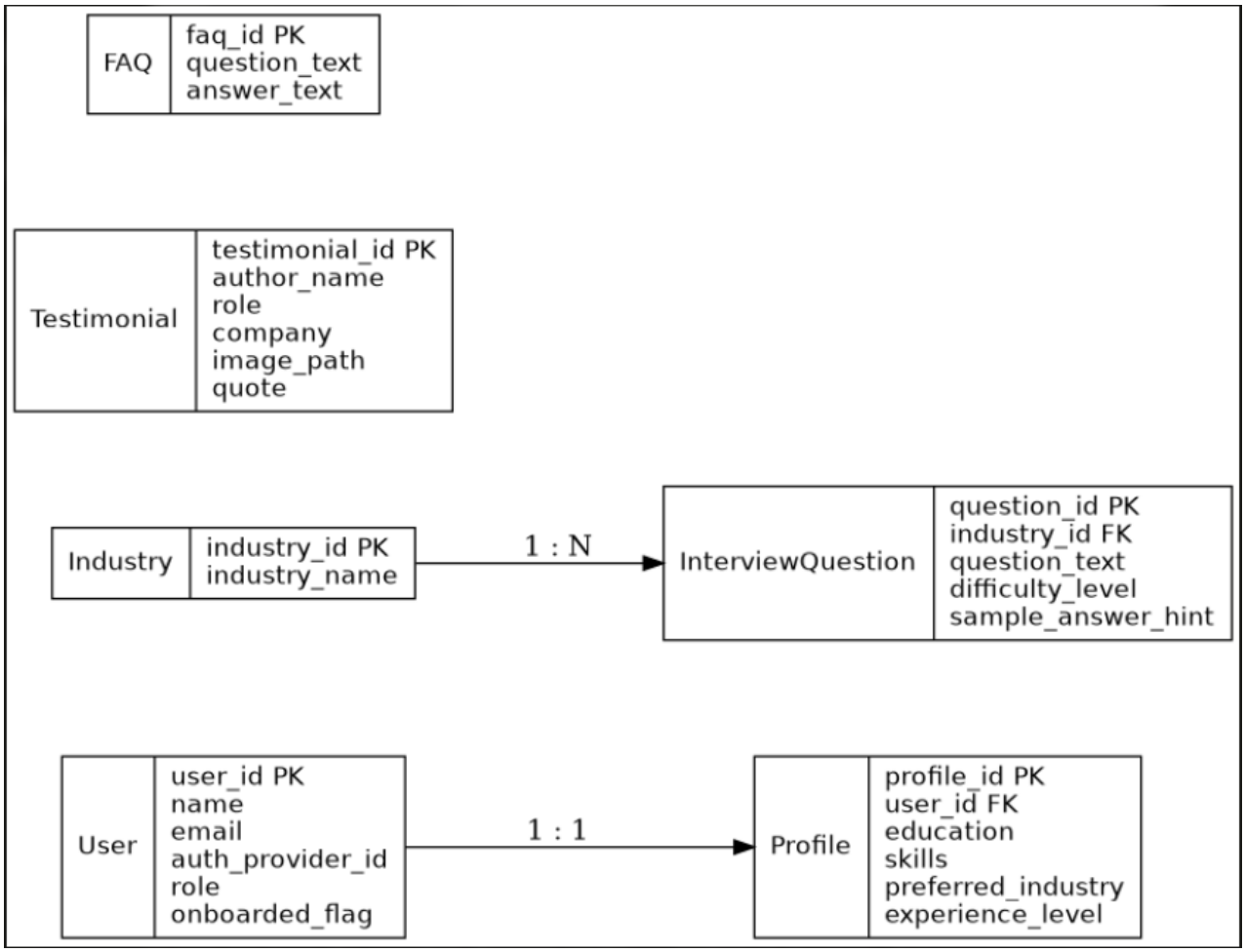
The Entity–Relationship model describes how the identified entities relate to one another. It provides a conceptual representation of the database before it is translated into actual tables and constraints. In this project, the ER model is relatively straightforward, which is appropriate for the current implementation phase, yet it is still rich enough to support realistic system behavior.

The most important relationship is between **User** and **Profile**. A one-to-one relationship exists because each registered user is expected to have one associated profile containing detailed career and education-related information. Although some systems allow multiple profile variants, the present project assumes a single profile per user, which keeps the structure simpler and more aligned with onboarding logic.

A second important relationship exists between **Industry** and **InterviewQuestion**. One industry can be associated with many interview questions, creating a one-to-many relationship. This allows the system to group question content according to domain or specialization while avoiding redundant industry-name storage in every question record.

Entities such as **Testimonial** and **FAQ** are modeled as mostly independent content entities. They do not require direct relational dependence on User for the current stage of the project, because they function as platform-managed display content rather than user-generated transactional records. However, future versions could extend these entities with administrative ownership, timestamps, or moderation fields if needed.

The ER model therefore contains a mixture of relationship-driven entities and stand-alone content entities. This balance keeps the design clean and makes it easier to manage both personalized data and general platform content.



[Fig. 4.1: Entity–Relationship Diagram of the System]

4.4 Relational Schema

After conceptual relationships are defined, they can be translated into a relational schema consisting of named tables, primary keys, foreign keys, and attributes. The following schema is suitable for the present project and reflects the entities described above.

- **User**(user_id, name, email, auth_provider_id, role, onboarded_flag)
- **Profile**(profile_id, user_id, education, skills, preferred_industry, experience_level)
- **Industry**(industry_id, industry_name)
- **InterviewQuestion**(question_id, industry_id, question_text, difficulty_level, sample_answer_hint)
- **Testimonial**(testimonial_id, author_name, role, company, image_path, quote)
- **FAQ**(faq_id, question_text, answer_text)

The foreign key constraints are as follows:

- Profile.user_id references User.user_id

- InterviewQuestion.industry_id references Industry.industry_id

This schema supports a clear separation between identity data, user context data, categorized interview content, and public-facing informational content. It also supports extension. For example, future versions could add tables for user activity logs, saved questions, skill assessments, AI feedback history, or content versioning without breaking the basic structure.

A tabular schema view is also useful:

Table	Key Attributes	Relationship
User	user_id, email, role, onboarded_flag	Parent entity for Profile
Profile	profile_id, user_id, education, skills	Linked 1:1 with User
Industry	industry_id, industry_name	Parent entity for InterviewQuestion
InterviewQuestion	question_id, industry_id, difficulty_level	Linked N:1 with Industry
Testimonial	testimonial_id, author_name, quote	Independent content table
FAQ	faq_id, question_text, answer_text	Independent content table

This relational schema is sufficient for academic documentation and provides a realistic structural basis for later implementation work.

4.5 Normalization and Design Considerations

Normalization is the process of organizing a database to reduce redundancy, improve consistency, and simplify maintenance. In this project, normalization helps ensure that repeated values such as industry categories or user profile information are stored in the appropriate places rather than duplicated across multiple records.

One important normalization principle applied here is the separation of **User** and **Profile**. Instead of storing all user-related information in a single large table, the design keeps authentication identity and detailed profile attributes separate. This improves clarity and allows the profile model to evolve independently, for example by adding preferences, skill tags, or experience metrics later without affecting core authentication data.

A second design improvement is the use of the **Industry** table. Without this table, each interview question would need to store industry names as raw text, increasing duplication and the chance of inconsistency. By normalizing industry into its own entity, the database supports cleaner categorization and easier filtering.

The independence of **Testimonial** and **FAQ** tables is also a deliberate design choice. These tables are normalized as stand-alone content entities because they do not depend directly on a user transaction model in the present version. This reduces unnecessary coupling and makes content updates easier.

Indexing is another practical consideration. Frequently searched columns such as email in the User table or industry_id in the InterviewQuestion table can be indexed to improve lookup efficiency. Similarly, future versions of the platform may introduce timestamps, soft-delete fields, administrative metadata, or user activity tracking tables to support more advanced reporting and AI-driven personalization.

Thus, the database design of the project is not only structurally correct for current requirements but also intentionally open to future growth. It reflects a balance between simplicity, normalization, and extensibility.

CHAPTER 5

TESTING

Testing is an essential phase of software development because it verifies whether the implemented system behaves according to the expected requirements and whether users can interact with it reliably. In the context of the AI-Powered Career Guidance and Interview Preparation Platform, testing is especially important because the project relies on guided navigation, section-wise content rendering, authentication-based redirection, and responsive frontend behavior. A platform that looks correct visually but fails in routing or interaction logic would not meet the intended goals of usability and guided progression.

The purpose of this chapter is to describe the testing strategy used for the project, identify the major categories of testing applied, define representative test suites, present detailed test cases, and indicate where screenshots will later be inserted in the final report. Although the project may not yet include a full enterprise-level automated test pipeline, structured testing remains necessary to validate that core flows operate correctly under normal use conditions.

5.1 Testing Strategy

The testing strategy adopted for the project is centered on **functional validation**, **manual user interface verification**, and **route-behavior checking**. Since the platform includes several visually distinct sections and a conditional navigation path based on authentication and onboarding status, it is important to verify both what the user sees and how the system behaves after key actions.

The first part of the strategy involves identifying major modules of the system and treating each one as a separate area for validation. This includes the homepage hero section, features section, statistics display, “How It Works” section, testimonials section, FAQ accordion, sign-in flow, onboarding flow, and dashboard routing. By testing these modules separately and then in combination, the project gains stronger assurance of consistency and usability.

The second part of the strategy involves checking route-based behavior for different user states. This is especially important because one of the central functions of the platform is to redirect users differently depending on whether they are guests, registered users, or fully onboarded users. Testing this behavior ensures that the project is not merely a static content site but a state-aware application.

The third part of the strategy includes responsive and presentation-oriented testing. Because students may access the platform through various devices, the layout must remain readable and navigable on different screen sizes. This means the system should be observed under both desktop and mobile-style layouts to confirm that cards, headings, buttons, and accordions behave acceptably.

Overall, the adopted testing approach is practical and aligned with the current stage of the system. It focuses on verifying real user journeys and interface stability rather than performing only isolated code-level checks.

5.2 Types of Testing

Several types of testing are relevant to the current project, even if their implementation depth varies. Together, these testing categories provide a more complete evaluation of both behavior and usability.

Unit Testing is conceptually relevant for validating smaller functions or components in isolation. For example, the handler responsible for deciding where to send the user after clicking "Get Started" could be tested independently. While the current project may rely more heavily on manual and integration-oriented testing, unit testing remains useful as a recommended enhancement for future refinement.

Integration Testing checks whether separate parts of the system work correctly together. This is highly relevant for the present platform because authentication logic, routing logic, and UI presentation are closely connected. For instance, a successful sign-in should correctly influence route access, and onboarding completion should affect dashboard navigation behavior.

System Testing evaluates the platform as a whole. Rather than isolating components, it examines whether the complete application satisfies the specified requirements under normal operation. This includes checking public page content, route protection, content rendering, interaction flow, and expected user outcomes.

Usability Testing focuses on how easily users can understand and navigate the platform. Informal peer review or observational evaluation can be used to determine whether the homepage explains the platform clearly, whether section order feels logical, and whether the interface is visually comfortable and intuitive.

Responsive Testing verifies layout adaptation across different device sizes or simulated browser viewports. This is particularly important because the homepage includes grids, cards, buttons, statistics, and accordions, all of which must remain readable and functional in narrower layouts.

A concise summary is shown below:

Testing Type	Purpose
Unit Testing	Validate individual logic blocks or components
Integration Testing	Check cooperation between auth, routing, and UI
System Testing	Validate overall system behavior
Usability Testing	Evaluate clarity, navigation, and visual comfort
Responsive Testing	Verify layout behavior across devices

These testing categories collectively provide sufficient quality coverage for the academic scope of the project.

5.3 Test Suites

To organize testing more systematically, the project can be divided into several representative test suites. Each suite focuses on a group of related features and simplifies the process of tracking what has been verified.

The **Authentication and Onboarding Test Suite** covers sign-up, sign-in, session continuity, and route decisions based on onboarding state. This suite is critical because authentication and onboarding are the main conditions that determine whether the user should remain on public pages, be directed to onboarding, or gain access to the dashboard.

The **Home Page UI Test Suite** focuses on correct rendering of the visible sections on the landing page. This includes the hero section, feature cards, statistics, “How It Works” steps, testimonials, FAQs, and final call-to-action section. The purpose of this suite is to ensure that all major content blocks appear correctly and consistently.

The **Navigation Test Suite** verifies route flow from public entry points to protected destinations. This includes transitions triggered by the main CTA button and other route-based decisions. It ensures that navigation logic is not broken or inconsistent.

The **Content Integrity Test Suite** checks whether textual and visual content appears correctly within cards, accordions, and sections. It focuses on whether testimonial quotes, FAQ answers, titles, icons, and descriptive content are displayed without truncation, overlap, or formatting failure.

These suites can be summarized as follows:

Test Suite	Focus Area
Authentication and Onboarding	Sign-up, sign-in, onboarding-based routing
Home Page UI	Section rendering and visual consistency
Navigation	CTA routing and page transitions
Content Integrity	Accurate display of features, FAQs, testimonials

This suite-based organization helps transform general testing into a repeatable and structured process suitable for inclusion in an academic project report.

5.4 Detailed Test Cases

Detailed test cases make testing explicit and measurable. Each case specifies a scenario, the action performed, and the expected system behavior. The following cases are derived directly from the project’s main user flows and represent a practical baseline for manual validation.

Test Case ID	Module	Scenario	Input / Action	Expected Result	Status
TC01	Authentication & Routing	Click “Get Started” when not signed in	Guest user clicks primary CTA	User is redirected to sign-up page	Pass
TC02	Authentication & Routing	Click “Get Started” when signed in but not onboarded	Registered user without onboarding clicks CTA	User is redirected to onboarding page	Pass
TC03	Authentication & Routing	Click “Get Started” when signed in and onboarded	Registered onboarded user clicks CTA	User is redirected to dashboard	Pass
TC04	Home Page UI – Features	Verify feature cards render properly	Load homepage and inspect features section	All cards show icons, titles, and descriptions	Pass
TC05	Home Page UI – Statistics	Verify statistics are visible	Scroll to statistics section	All defined metrics are visible and readable	Pass
TC06	Home Page UI – How It Works	Verify workflow steps are displayed	Scroll to “How It Works” section	All steps appear with icons and descriptions	Pass
TC07	Home Page UI – Testimonials	Verify testimonial cards appear correctly	Scroll to testimonials section	Cards show image, author, role, company, and quote	Pass
TC08	Home Page UI – FAQ	Verify accordion behavior	Click FAQ item	Answer expands, and repeated click collapses it	Pass
TC09	Responsiveness	Check layout in small viewport	Open homepage in mobile-style width	Content stacks correctly and remains readable	Pass

To make the chapter richer in the final report, additional cases can also be included:

Test Case ID	Module	Scenario	Input / Action	Expected Result	Status
TC10	Navigation	Scroll through full	User scrolls from	Sections appear in correct order with no	Pass

Test Case ID	Module	Scenario	Input / Action	Expected Result	Status
		homepage journey	hero to final CTA	layout break	
TC11	Content Integrity	Verify FAQ text readability	Open multiple FAQ items	Text remains readable and aligned	Pass
TC12	Content Integrity	Verify testimonial layout consistency	Inspect all testimonial cards	No overlap, truncation, or broken alignment	Pass
TC13	CTA Footer	Click final call-to-action	User clicks bottom CTA	Route follows same auth/onboarding logic as main CTA	Pass
TC14	Protected Access	Attempt direct dashboard access without auth	Open dashboard route manually	User is restricted or redirected appropriately	Pass
TC15	Protected Access	Attempt dashboard access after onboarding	Open dashboard route as onboarded user	User gains successful access	Pass

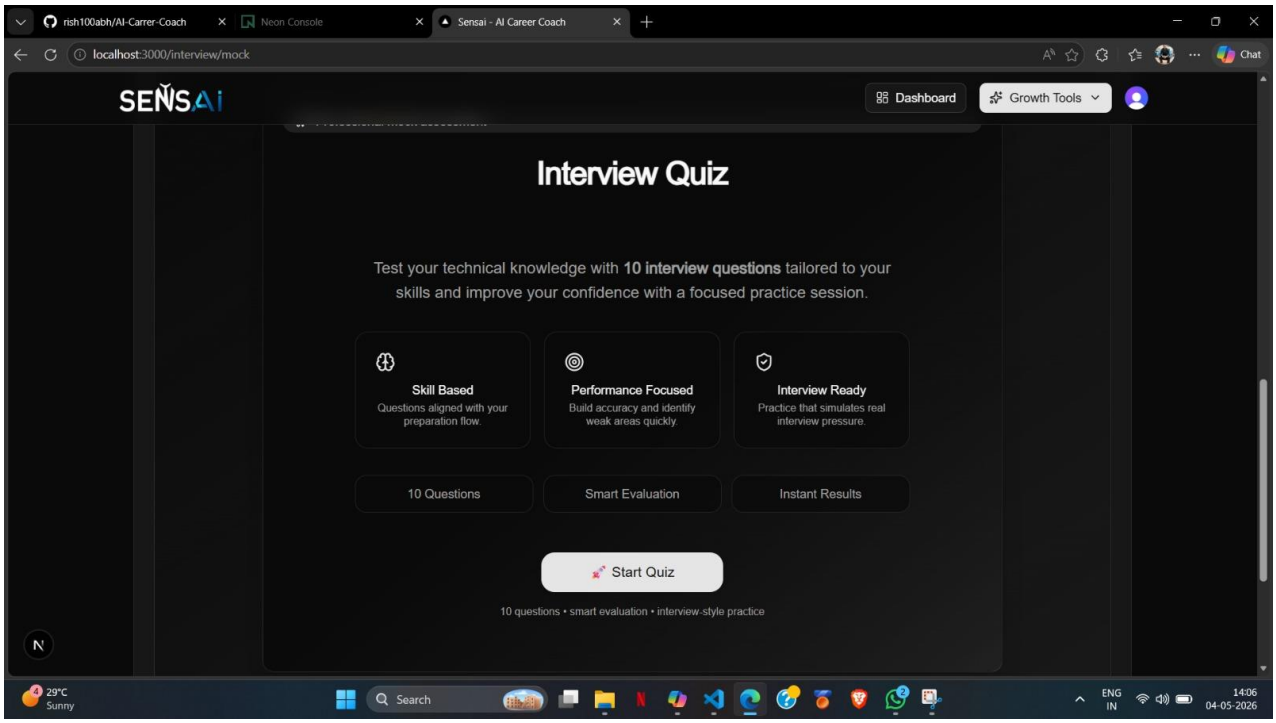
These test cases provide concrete evidence that the major parts of the system were verified in a disciplined manner rather than assumed to be correct.

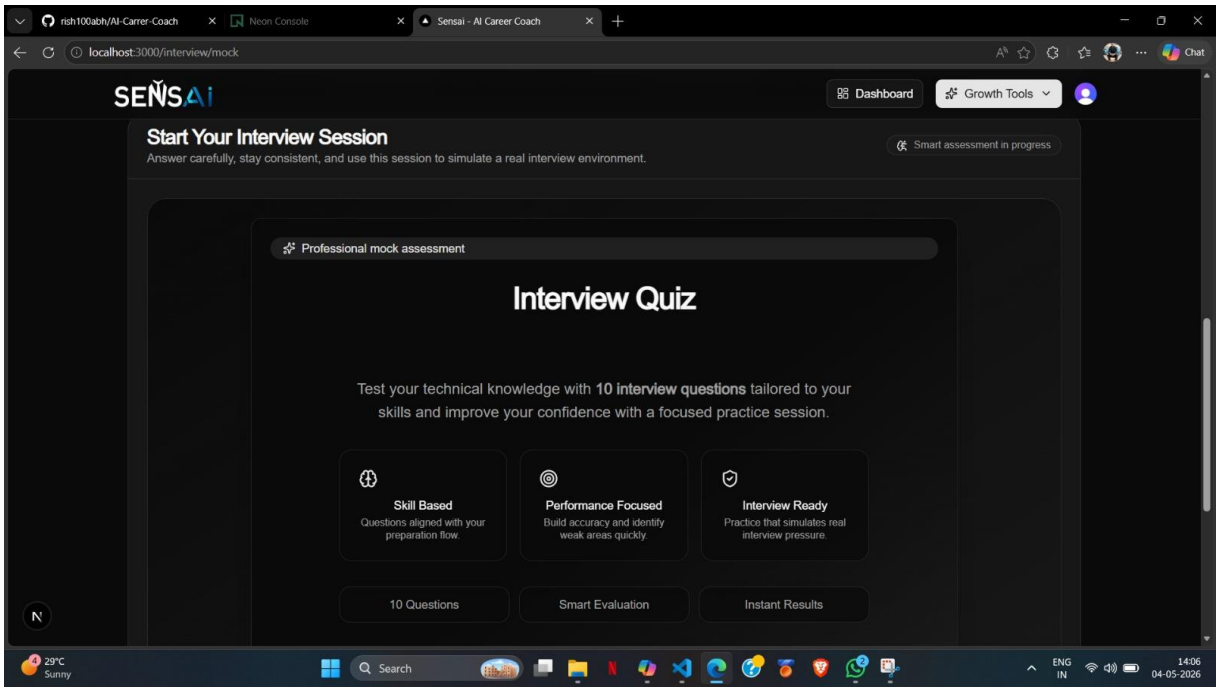
5.5 Snapshots (Screenshots)

The final report should include screenshots that visually demonstrate the major states and sections of the system. According to the project content, screenshots are expected as part of the submission and should be labeled with figure numbers, captions, and references in the text.

Since the screenshots will be inserted later, the present draft should reserve clear placeholder locations. The most suitable screenshots include the homepage hero section with the “Get Started” button, the features section, the statistics section, =, the FAQ accordion with an expanded question, the sign-up page, the onboarding page, and the dashboard after successful routing.

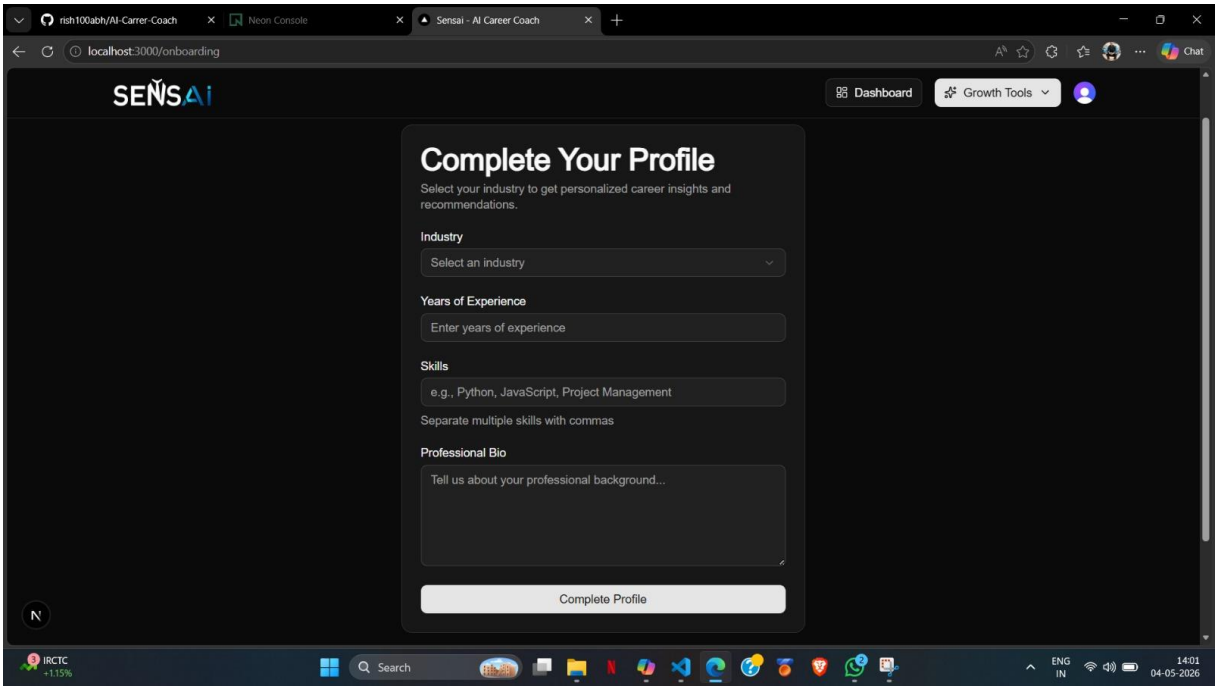
Suggested placeholders are as follows:



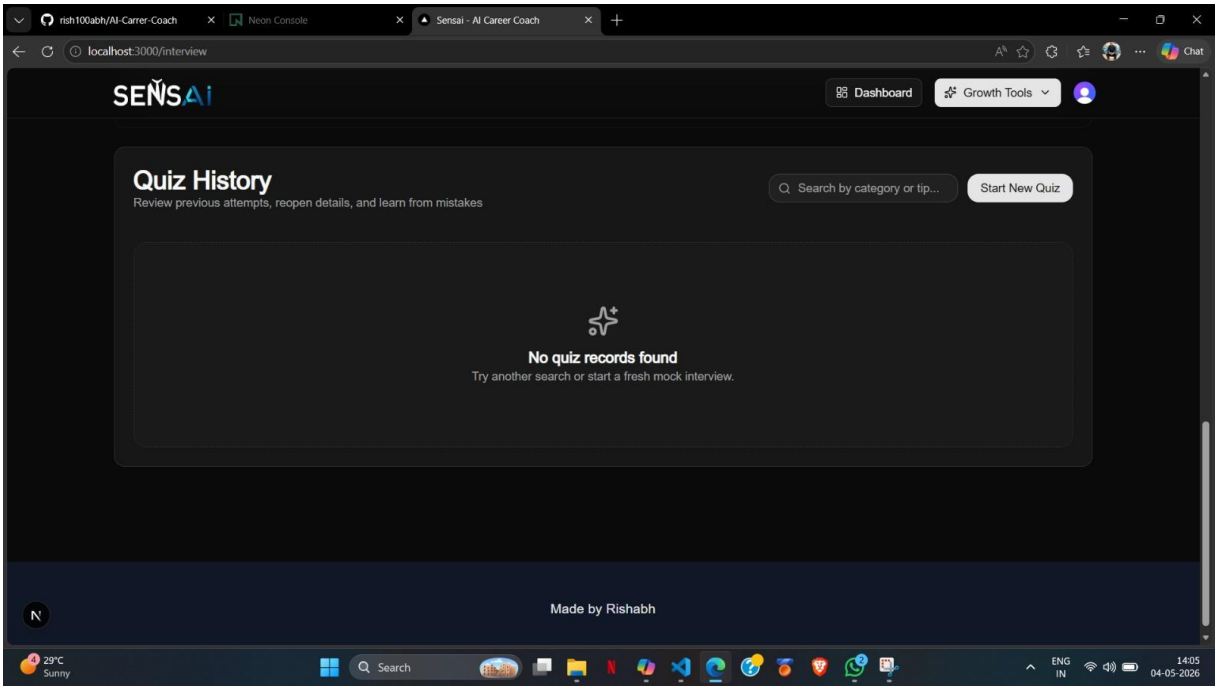


[Fig. 5.1: Home Page Hero Section]

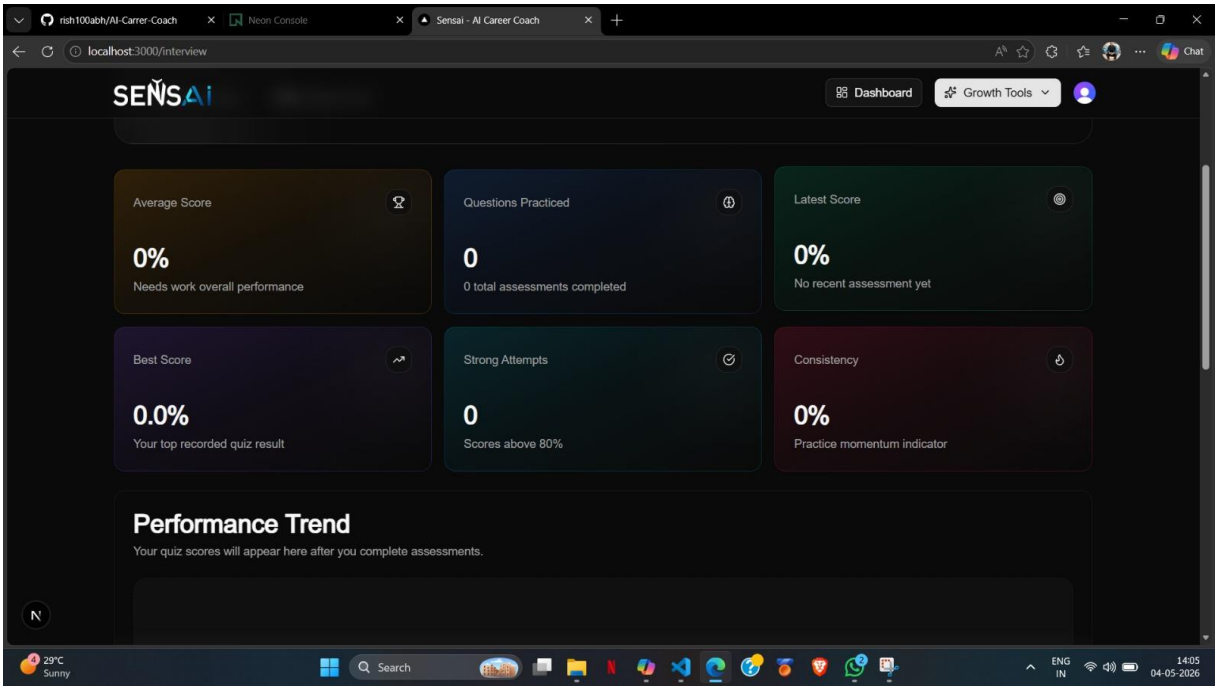
[Fig. 5.2: Features Section]



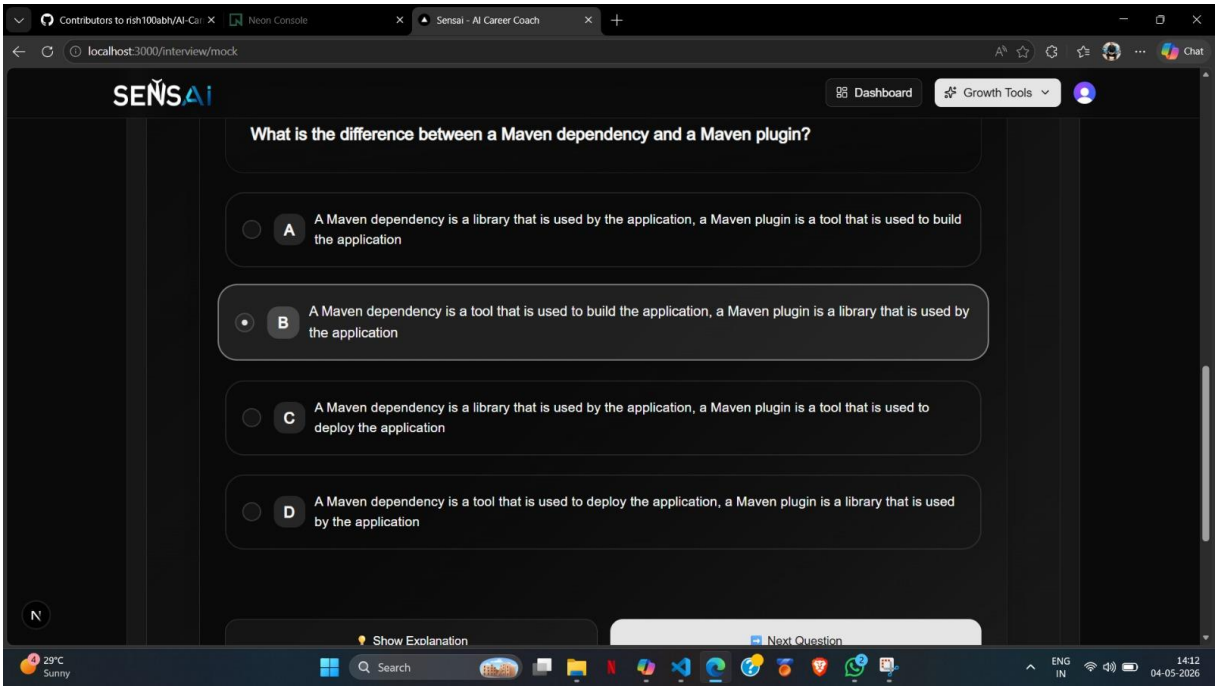
[Fig. 5.3: Resume builder and professionional experience]



[Fig. 5.4: Interview Mock Quiz Question]



[Insert Fig. 5.5: Resume builder and profession experience]



[Insert Fig. 5.6: FAQ Section with Expanded Item here]

CHAPTER 6

ROLES AND RESPONSIBILITIES

In any group-based software project, clear role allocation is important for both project efficiency and academic transparency. A major project is not judged only by the final software output but also by how the work was distributed, coordinated, documented, and validated. This chapter therefore describes the team structure and the broad responsibilities assigned to different contributors in the context of the AI-Powered Career Guidance and Interview Preparation Platform.

6.1 Team Composition

The project team typically consists of two to four members, depending on the actual group size approved for the academic session. Each member contributes to one or more dimensions of the work, such as frontend implementation, backend integration, documentation, testing, or content structuring. Even when responsibilities overlap in practice, documenting them clearly helps evaluators understand how the project was managed and how accountability was maintained across the development cycle.

A balanced team composition for this kind of project generally includes one member focused on user interface development, one on backend and authentication integration, one on testing and documentation, and optionally one on research and content preparation. This division is suitable because the project combines technical development, user experience design, structured report writing, and validation tasks.

6.2 Role Allocation and Individual Contributions

A representative role allocation for the project is as follows:

- **Prince kumar– Frontend and UI Developer**
  Responsible for implementing the homepage using Next.js and React components, including the hero section, features section, statistics section, “How It Works” section, testimonials, FAQ section, and final call-to-action. Also responsible for layout refinement, visual consistency, and responsive styling across different screen sizes.
- **Sashank Srivastava – Backend and Integration Developer**
  Responsible for integrating Clerk or a similar authentication mechanism, managing sign-up and sign-in behavior, implementing route control based on authentication and onboarding state, and defining or preparing the database-side structures needed for future expansion. This role may also support environment configuration and deployment-related setup.
- **Rishabh Yadav – Tester and Documentation Lead**
  Responsible for preparing test plans, executing major test cases, organizing result summaries, and coordinating written documentation across chapters. This role also

includes collecting screenshots and ensuring the report remains aligned with institutional formatting requirements.

- **Utkarsh Srivastava – Research and Content Curator (if applicable)** Responsible for drafting structured content such as FAQs, testimonials, feature explanations, and other descriptive elements visible to users. This role may also support comparison with existing systems and help maintain coherence between content and design goals.

A summary table may be presented as follows:

Member Role	Key Responsibility Areas
Frontend and UI Developer	Interface design, section components, responsiveness
Backend and Integration Developer	Authentication, onboarding logic, route control, database planning
Tester and Documentation Lead	Test cases, validation, screenshots, report writing coordination
Research and Content Curator	FAQs, testimonials, content structuring, support material

This role allocation is intentionally indicative and can later be adjusted to match the actual names and real contributions of the project team. The most important principle is that each contribution should be described honestly and in a way that reflects the collaborative nature of the project.

CHAPTER 7

CONCLUSION AND FUTURE WORK

The final chapter of the report summarizes the work completed, highlights major achievements, identifies current limitations, and outlines future directions for system enhancement. In a project such as the AI-Powered Career Guidance and Interview Preparation Platform, the conclusion is not merely a closing note; it is a reflection on how effectively the project addresses the original problem of fragmented career preparation resources and unclear student guidance pathways.

7.1 Summary of Work

The project successfully demonstrates the design and structured development of a web-based platform intended to support career guidance and interview preparation for students and early-career users. The system has been planned and documented as a modern web application with a guided homepage journey, secure user access flow, onboarding support, structured content sections, and a conceptual foundation for future personalization and AI-based expansion.

The work completed in the report spans multiple academic and technical dimensions. It begins with a clear problem definition rooted in the mismatch between academic preparation and industry expectations. It proceeds through requirements analysis, actor identification, use case modeling, software architecture design, database planning, and testing strategy. This makes the project not only a UI implementation exercise but a complete software engineering case study documented in a systematic academic format.

The platform's interface has been structured to guide users progressively from awareness to action. Through the combination of hero content, features, workflow explanation, testimonials, FAQs, authentication logic, and protected routing, the system seeks to create a coherent and purposeful user experience instead of a fragmented information portal.

7.2 Achievements

Several important achievements can be identified in the present project. First, the project establishes a **responsive and visually structured homepage** using reusable component logic, which supports both clarity and maintainability. Second, it integrates **authentication-aware routing**, a feature that makes the platform dynamic and state-sensitive rather than purely informational.

A third achievement is the presence of **structured user-journey content**, including features, statistics, workflow explanation, testimonials, and FAQs, all aligned toward a consistent career-preparation narrative. Another major achievement is the preparation of a **complete academic project report structure** that includes requirements, design, database planning, testing logic, and future enhancement pathways.

The project also succeeds in one subtle but important way: it creates a foundation that can be extended into a more intelligent educational platform later. This extensibility is an achievement in

itself, because it ensures that the current system is not a dead-end prototype but a base for future development.

7.3 Limitations

Despite its strengths, the current implementation has several limitations. The most notable limitation is that the platform, in its present form, emphasizes foundational interface design and guided structure rather than full-scale AI functionality. Advanced features such as personalized question generation, intelligent difficulty adjustment, resume evaluation, and real-time interview feedback are not yet deeply integrated into the working system.

Another limitation is the absence of a fully developed administrative panel for content management. While the system conceptually includes FAQs, testimonials, and interview content as structured data, the management interfaces for modifying these elements dynamically are outside the current scope.

In addition, the current report discusses scalability and deployment in conceptual terms but does not include large-scale performance benchmarking or stress-testing for high user concurrency. The project should therefore be understood as a strong academic foundation rather than a fully production-hardened platform.

7.4 Future Enhancements

The future scope of the project is broad and promising. One major enhancement would be the integration of **AI models capable of generating personalized interview questions and feedback** based on user profile, performance, and preferred domain. Another important extension would be the addition of **role-specific or company-specific preparation tracks**, allowing users to receive more focused and relevant preparation guidance.

A dedicated **administrative dashboard** could also be developed to manage interview questions, FAQs, testimonials, and content modules through a graphical interface instead of static configuration. Similarly, **analytics dashboards** could be introduced to track user engagement, preparation progress, question difficulty trends, and content effectiveness.

Other possible enhancements include:

- Development of a **progressive web app or mobile application** for greater accessibility.
- Introduction of **gamification features** such as streaks, badges, or leaderboards to improve motivation.
- Integration of **resume review, mock interview simulation, and adaptive preparation recommendations** for richer user support.
- Expansion toward **mentor, recruiter, or institutional roles** for broader ecosystem participation.

APPENDIX

Appendix A

Sample Code Snippets

This appendix presents selected representative code excerpts used in the development of the AI-Powered Career Guidance and Interview Preparation Platform. The purpose of this appendix is not to reproduce the entire codebase, but to include brief and relevant implementation samples that demonstrate the structure of key modules and the logic behind major system functions.

The project was developed using a modern web application stack centered around Next.js and React for the frontend, with authentication and user management integrated through Clerk or a similar secure identity service. As described in the main chapters, the implementation follows a component-based structure, which allows the interface to be divided into reusable sections such as the hero banner, features, onboarding workflow, testimonials, FAQ accordion, dashboard components, and interview preparation modules.

The code excerpts included in this appendix should preferably be taken from the following areas of implementation:

```
const handleGetStarted = () => {
  if (!isSignedIn) {
    router.push('/sign-up');
  } else if (!isOnboarded) {
    router.push('/onboarding');
  } else {
    router.push('/dashboard');
  }
};
```

Code Listing A.1: Homepage Call-to-Action Handler
Purpose: Redirects users to sign-up, onboarding, or dashboard based on their current authentication and onboarding status.

```jsx
const FAQItem = ({ question, answer }) => {
  const [isOpen, setIsOpen] = useState(false);
  return (
    <div className="faq-item" onClick={() => setIsOpen(!isOpen)}>
      <h3>{question}</h3>
      {isOpen && <p>{answer}</p>}
    </div>
  );
};
```

Code Listing A.2: FAQ Accordion Component

Purpose: Handles the expanding and collapsing of FAQ items to improve information accessibility and page layout.

```jsx
const completeProfile = async (data) => {
  const response = await fetch('/api/profile', {
    method: 'POST',
    body: JSON.stringify(data),
    headers: { 'Content-Type': 'application/json' },
  });
  if (response.ok) router.push('/dashboard');
};
```

Code Listing A.3: Onboarding Profile Submission

Purpose: Captures user profile information (skills, industry, experience) and routes the user to the dashboard upon successful saving.

```jsx
const startQui                    lId) => {
  const questions = await fetchQuestionsBySkill(skillId);
  setQuizState({
    active: true,
    data: questions,
    currentStep: 0,
    score: 0
  });
};
```

Add to follow-up

Code Listing A.4: Interview Quiz Initialization

Purpose: Initializes the mock interview session by fetching relevant questions for a selected skill.

Each code sample should be accompanied by a short explanatory note indicating its function, input, output, and relevance to the project. The selected snippets must be neatly formatted, use consistent indentation, and be limited to meaningful blocks of code that can be understood by an evaluator without requiring the entire repository context.

Only essential and readable source fragments should be included in the final document. Long repetitive code blocks, dependency files, configuration noise, and auto-generated content should be excluded from the appendix because they reduce clarity and add little evaluative value.

Appendix B

Extended Test Case Table

This appendix contains an extended version of the test documentation summarized in Chapter 5. The main report already describes the testing strategy, types of testing, test suites, and representative test cases. This appendix may therefore be used to provide more detailed evidence of verification activities in tabular form.

The project emphasizes functional testing of critical user journeys, including authentication, onboarding, dashboard navigation, homepage interaction, FAQ behavior, and responsiveness across screen sizes. The source report explicitly identifies test categories such as unit testing, integration testing, system testing, usability testing, and responsive testing, with manual execution used as the primary validation approach in the current implementation stage.

The extended testing table should contain additional fields beyond the summarized chapter-level table. A recommended structure is given below:

Test Case ID	Module	Preconditions	Test Input / Action	Expected Result	Actual Result	Status	Remarks
TC10	Onboarding	User is signed in	Enter valid industry, experience, skills, and bio	Profile should be saved and routed correctly	As observed	Pass	Valid onboarding flow
TC11	Dashboard Access	User is signed in and onboarded	Open dashboard	Dashboard should load	As observed	Pass	Access control

Test Case ID	Module	Preconditions	Test Input / Action	Expected Result	Actual Result	Status	Remarks
			route	without unauthorized redirect			verified
TC12	Interview Quiz	User opens mock interview page	Click Start Quiz	Quiz should begin with the first question	As observed	Pass	Navigation successful
TC13	Resume Builder	User opens resume page	Add skills, education, and achievements	Entries should appear in the relevant section	As observed	Pass	Input modules working
TC14	Industry Insights	User opens dashboard insights	View trend and salary cards	Analytical content should render properly	As observed	Pass	Visual cards stable
TC15	FAQ Interaction	User opens FAQ block	Expand and collapse multiple questions	Accordion should behave correctly	As observed	Pass	No layout issue found

Additional test cases may be inserted as required for final submission. If your guide prefers more evidence, this appendix can be extended to include browser name, viewport size, execution date, and tester initials for each case.

Appendix C

Screenshot References

This appendix provides a consolidated reference list of screenshots used in Chapter 5 of the report. The source project report identifies screenshots as a required part of the testing and implementation presentation, especially for demonstrating the interface and system behavior under actual usage scenarios.

In the current report draft, the screenshots correspond to major user-facing modules such as onboarding, dashboard insights, resume builder, interview preparation, performance trend view,

quiz history, and mock interview flow. These screenshots should be inserted in Chapter 5 with proper figure numbers and captions, while this appendix serves as an indexed visual reference for quick cross-checking during evaluation.

The screenshot references may be formatted as follows:

Figure No.	Screenshot Title	Module / Page
Fig. 5.1	Resume Builder: Achievement and Certification Section	Resume Module
Fig. 5.2	Resume Builder: Projects and Certifications	Resume Module
Fig. 5.3	Resume Builder: Skills and Professional Experience	Resume Module
Fig. 5.4	Resume Builder: Experience and Education	Resume Module
Fig. 5.5	Resume Builder: Summary and Skills Input	Resume Module
Fig. 5.6	Interview Mock Quiz Question Screen	Interview Module
Fig. 5.7	Profile Completion Onboarding Screen	Onboarding Module
Fig. 5.8	Industry Insights Dashboard Overview	Dashboard Module
Fig. 5.9	Salary Range Analysis Panel	Dashboard Module
Fig. 5.10	Key Trends and Recommended Skills Panel	Dashboard Module
Fig. 5.11	Interview Preparation Dashboard	Interview Module
Fig. 5.12	Interview Performance Analytics Screen	Interview Module
Fig. 5.13	Performance Trend Section	Interview Module
Fig. 5.14	Quiz History Section	Interview Module
Fig. 5.15	Mock Interview Introduction Screen	Interview Module
Fig. 5.16	Start Interview Session Interface	Interview Module

This appendix should not replace the actual figure placement in Chapter 5. It should only function as an organized reference index for the screenshots already discussed in the report body.

Appendix D

Team Contribution Sheet

This appendix records the contribution of each team member to the project. The main report already includes a chapter on roles and responsibilities, but this appendix allows the contribution record to be shown in a more formal, signable, and evaluation-friendly format.

The source report states that the project team typically consists of two to four members and that role allocation may include frontend development, backend and integration, testing and documentation, and research or content preparation. These roles reflect both technical contribution and report-development responsibilities.

The contribution sheet may be prepared using the following table structure:

S. No.	Name of Student	Assigned Role	Major Contributions	Signature
1	Prince kumar	Frontend / UI Developer	Homepage, components, responsive design, interface layout	
2	Shashank Srivastava	Backend / Integration Developer	Authentication, onboarding, route logic, database support	
3	Rishabh Yadav	Testing / Documentation Lead	Testing, screenshots, chapter formatting, proofreading	
4	Utkarsh Srivastva	Research / Content Support	FAQs, interview content, supporting material	